# Intelligent Support Ticket Classification with RAG


## Team Members:

1. Rahiq Khaled
2. Aya abdelshafy
3. Aya Elsaid
4. Radwa Ahmed
5. Tasneem Khaled


## 1. Introduction and Overview

Customer support is an essential service for organizations, but handling a large number of customer requests manually can be slow and time-consuming. Support agents must analyze each ticket, determine its category, assign it to the appropriate department, estimate its priority, and prepare a response. As the number of tickets increases, this process becomes less efficient and may lead to longer response times and inconsistent customer support.

This project presents an **Intelligent Support Ticket Classification System with Retrieval-Augmented Generation (RAG)**. The system automatically classifies incoming support tickets, retrieves similar historical cases using semantic search, and generates AI-assisted responses. The complete solution is deployed using Azure services and integrated into a web application that allows customers to submit tickets and administrators to manage reopened or unresolved cases.

### 1.1 Project Idea and Problem Definition

The main idea of this project is to automate the customer support workflow using Artificial Intelligence and Natural Language Processing techniques.

When a customer submits a support ticket, the system predicts its **type**, **support queue**, and **priority**. It then retrieves semantically similar historical tickets and uses a Large Language Model (Qwen) to generate an appropriate response. This approach reduces manual effort, improves response consistency, and provides customers with faster assistance.

The project addresses the following challenges:

- Manual ticket classification is time-consuming.
- Routing tickets to the correct department requires human effort.

- Writing responses manually increases response time.
- Traditional language models may generate responses without considering previous support cases.

## 1.2 Project Objectives

The objectives of this project are to:

- Develop an AI system for automatic support ticket classification.
- Predict ticket type, support queue, and priority.
- Retrieve similar historical tickets using semantic search.
- Generate context-aware responses using Retrieval-Augmented Generation (RAG).
- Deploy the classification model using Azure Machine Learning.
- Integrate all components through a FastAPI backend.
- Build a web application for customers and administrators.

## 1.3 Project Scope

The project consists of four main stages:

- **Milestone 1:** Data collection, preprocessing, exploratory data analysis, and feature engineering.
- **Milestone 2:** Development and evaluation of machine learning models for ticket classification.
- **Milestone 3:** Deployment of the classification model on Azure Machine Learning, integration with Azure AI Search, backend development, database implementation, and web application.
- **Milestone 4:** Model monitoring, experiment tracking, and preparation for future retraining.

---

# 2. Milestone 1 – Data Collection and Preprocessing

## 2.1 Objective

The objective of this milestone was to prepare a dataset suitable for machine learning and Retrieval-Augmented Generation (RAG). This involved exploring the raw dataset, cleaning missing values, removing redundant information, creating train/validation/test splits, generating text representations, and ensuring that no information leakage existed between the training and testing datasets.

## 2.2 Dataset Description

The project uses a customer support ticket dataset containing support requests from multiple departments. Each ticket includes textual information together with categorical labels describing the ticket.

| Property | Value |
|---|---|
| Total samples | 28,587 |
| Languages | English and German |
| Features | 16 |
| Ticket Types | 4 |
| Queues | 10 |
| Priorities | 3 |

The important attributes used throughout the project are:

| Column | Description |
|---|---|
| Subject | Ticket title |
| Body | Ticket description |
| Type | Ticket type label |
| Queue | Department responsible |
| Priority | Ticket priority |
| Answer | Historical response (used later only for reference) |
| Tags | Ticket metadata |

The target variables used for classification are:

- Type
- Queue
- Priority

## 2.3 Exploratory Data Analysis (EDA)

Before preprocessing, exploratory data analysis was conducted to understand the dataset characteristics and identify potential issues.

The analysis included:

- Missing value analysis
- Duplicate detection
- Language distribution
- Ticket type distribution
- Queue distribution
- Priority distribution

- Ticket length analysis

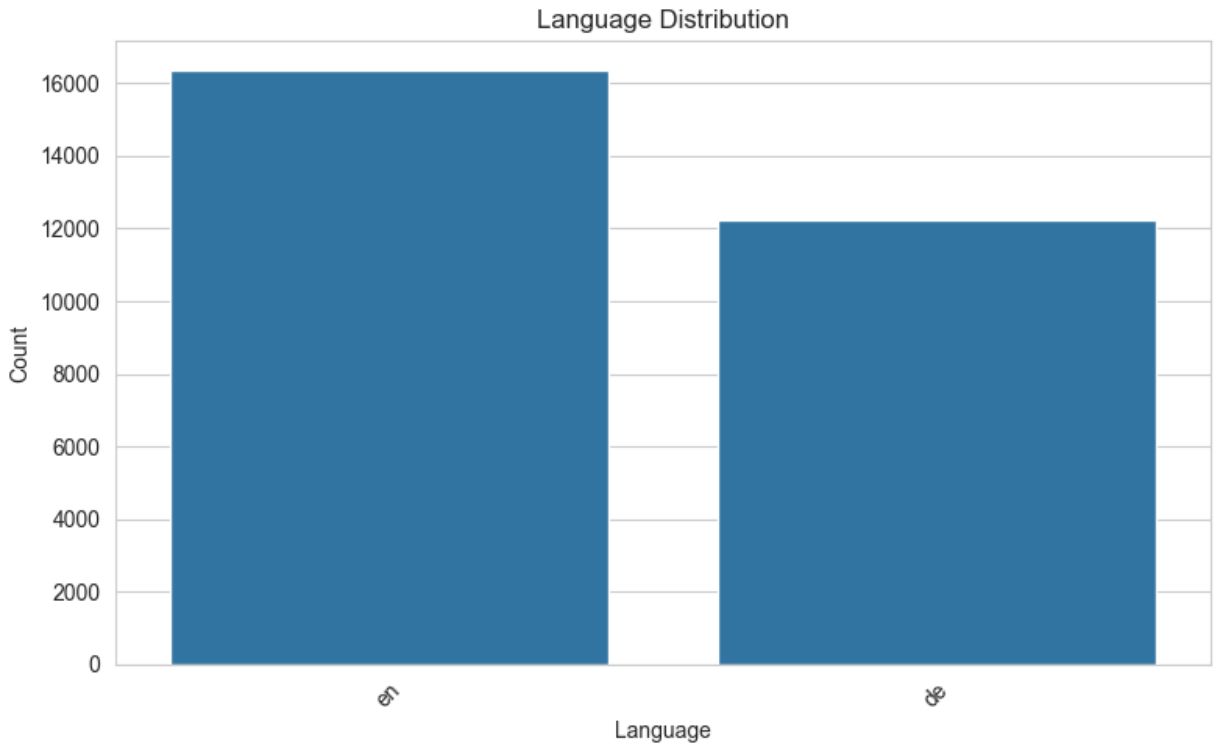
## Findings

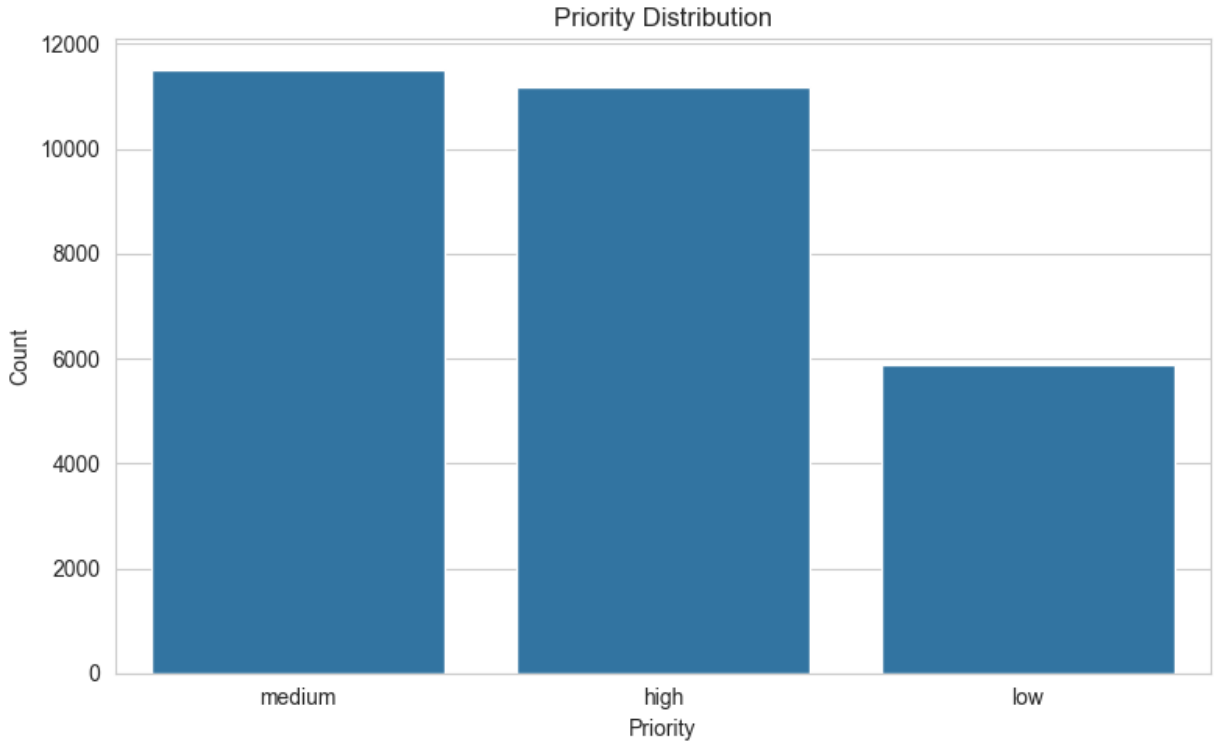
The raw dataset contained:

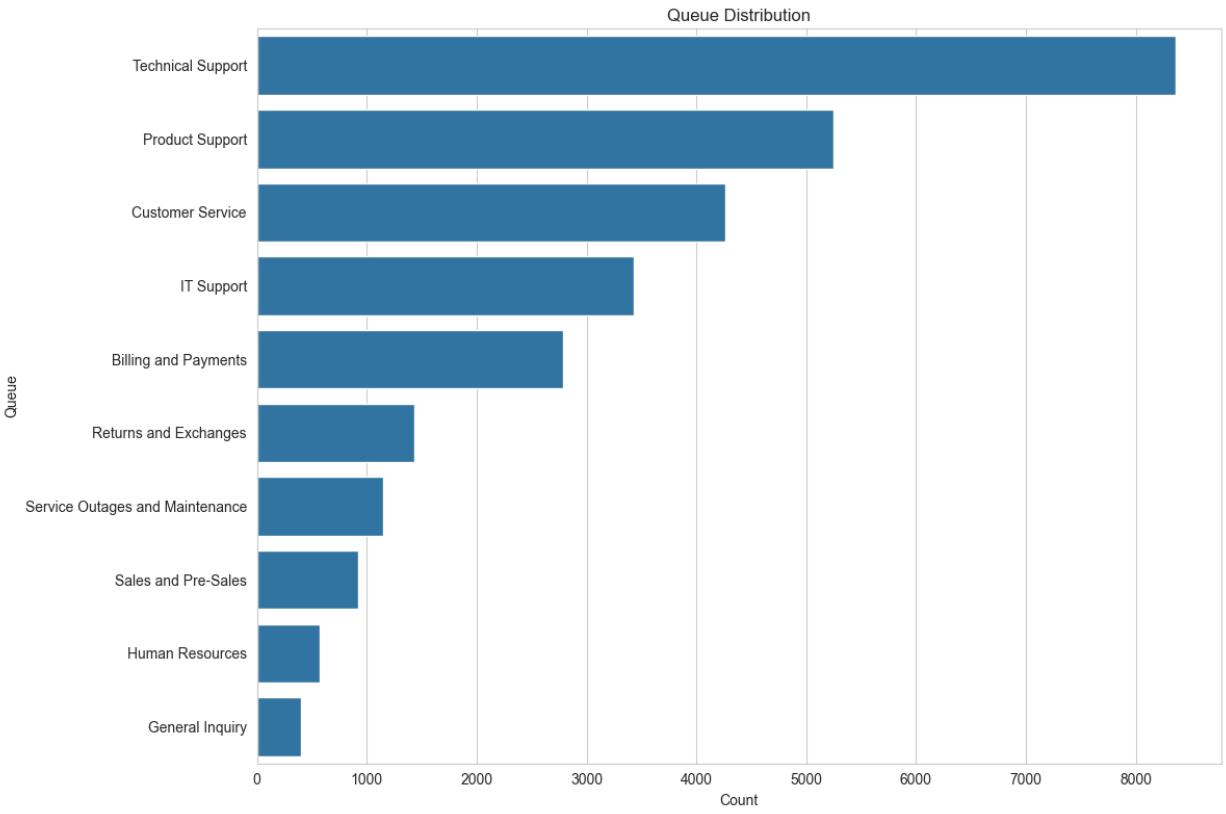
- 28,587 support tickets.
- Two languages (English and German).
- No duplicate tickets based on the combination of subject and body.
- Significant missing values in Tag 5–Tag 8.
- Subject field contained approximately 13% missing values.
- Answer field had only a few missing values.

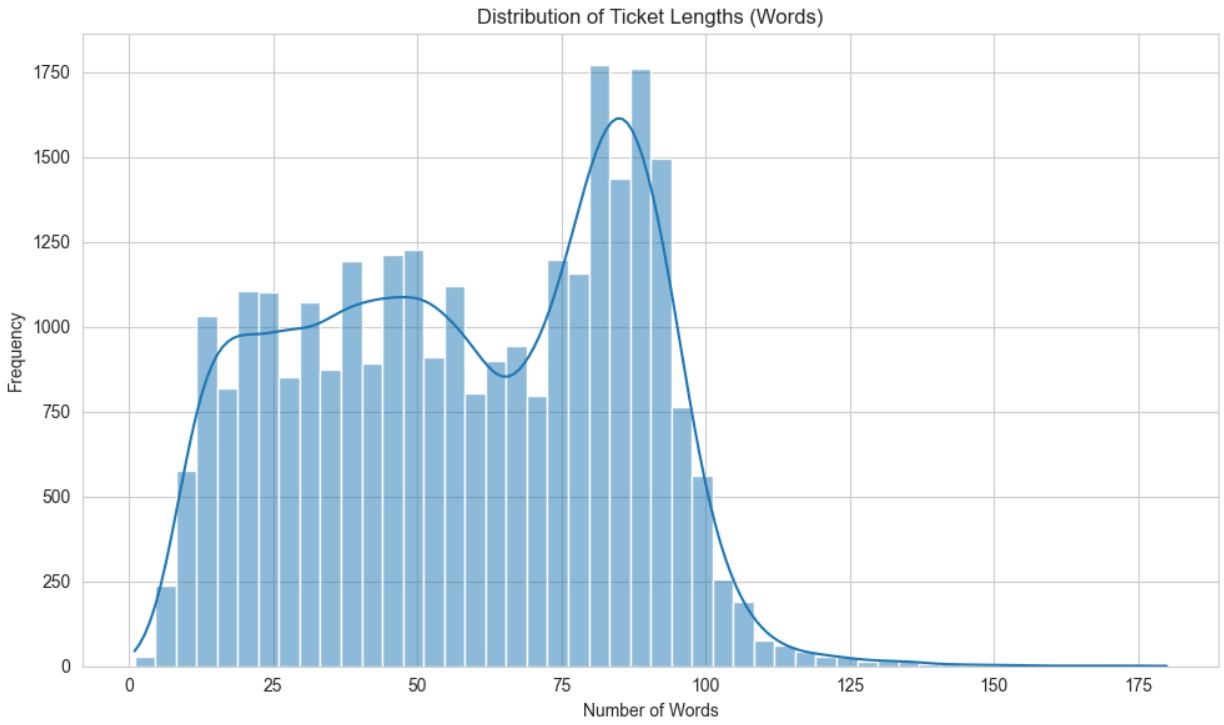
Include these figures here:

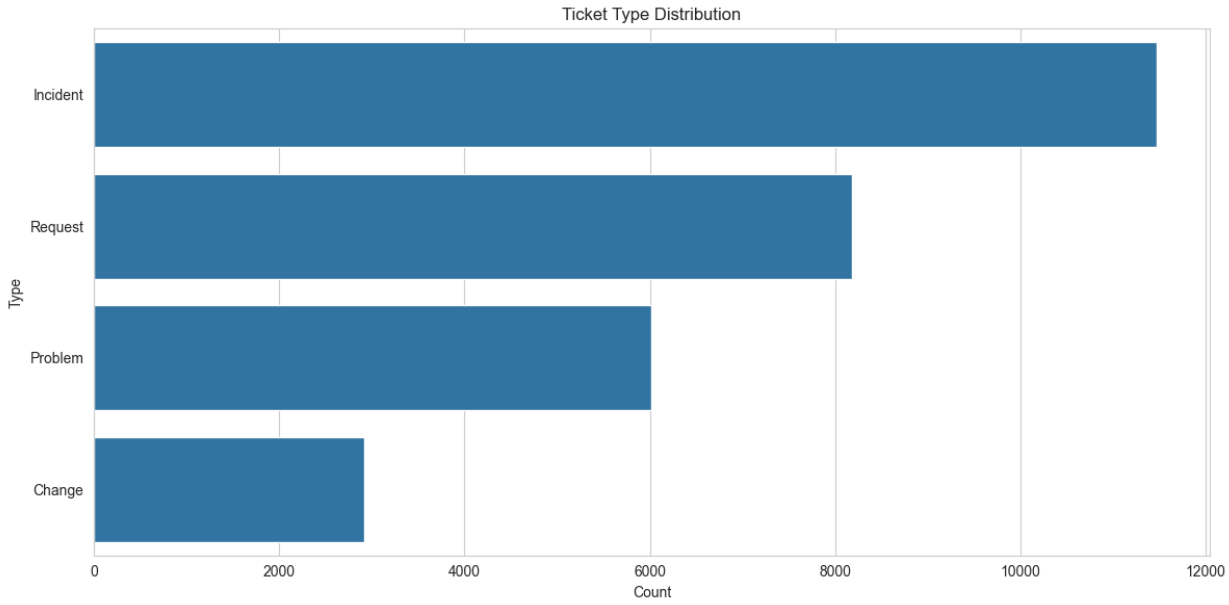
- Language Distribution
- Ticket Type Distribution
- Queue Distribution
- Priority Distribution
- Ticket Length Distribution


Language Distribution
16000
14000
12000
10000
8000
6000
4000
2000
0
Count
en
de
Language


Priority Distribution
12000
10000
8000
6000
4000
2000
0
Count
medium
high
low
Priority


Queue Distribution
Technical Support
Product Support
Customer Service
IT Support
Billing and Payments
Returns and Exchanges
Service Outages and Maintenance
Sales and Pre-Sales
Human Resources
General Inquiry
Queue
0
1000
2000
3000
4000
5000
6000
7000
8000
Count


Distribution of Ticket Lengths (Words)
Frequency
0
250
500
750
1000
1250
1500
1750
0
25
50
75
100
125
150
175
Number of Words




## 2.4 Data Cleaning

Several preprocessing operations were applied to improve data quality.

### Language Filtering

Only English tickets were retained because all downstream models were trained using English text.

- English tickets retained: **16,338**
- German tickets removed: **12,249**

---

### Missing Values

Instead of deleting rows unnecessarily, different strategies were applied depending on the feature.

- Missing subjects were replaced with empty strings because the body already contained sufficient information.
- Missing answers were replaced with empty strings since answers were not used as input features.
- Missing values in Tag 2–Tag 4 were replaced with **"none"** to indicate the absence of additional tags.

### Sparse Feature Removal

Tag 5–Tag 8 contained between 49% and 98% missing values.

Rather than imputing unreliable values, these columns were removed completely since they contributed very little useful information.

---

### Removing Unused Features

The following columns were removed:

- language
- version

After filtering to English, the language column contained only one value, making it uninformative for machine learning.

---

### Duplicate Detection

An additional duplicate check using the combination of subject and body confirmed that no exact duplicate tickets existed.

## 2.5 Text Preprocessing

The textual data was standardized before model training.

The preprocessing pipeline included:

- whitespace normalization
- removal of unnecessary line breaks
- concatenating subject and body into one text feature
- removing empty tickets
- generating character length
- generating word count

The final text field was created as

**Text = Subject + Body**

which served as the primary input to all classification models.

## 2.6 Near-Duplicate Removal

One challenge observed in the dataset was the presence of template-generated tickets that were almost identical but not exact duplicates.

To avoid data leakage between the training and testing sets, semantic duplicate removal was performed.

Sentence-BERT (all-MiniLM-L6-v2) was used to generate sentence embeddings for every ticket.

Cosine similarity was then computed between tickets.

Tickets with similarity greater than **0.95** were considered semantic duplicates.

| **Before** | **After** |
|---|---|
| 16338 | 10893 |

Removed tickets:

**5445 (33.3%)**

This step significantly reduced redundancy and improved the reliability of later evaluation.

## 2.7 Train / Validation / Test Split

After preprocessing, the cleaned dataset was divided into three subsets.

| **Split** | **Size** |
|---|---|
| Training | 7625 |
| Validation | 1634 |
| Testing | 1634 |

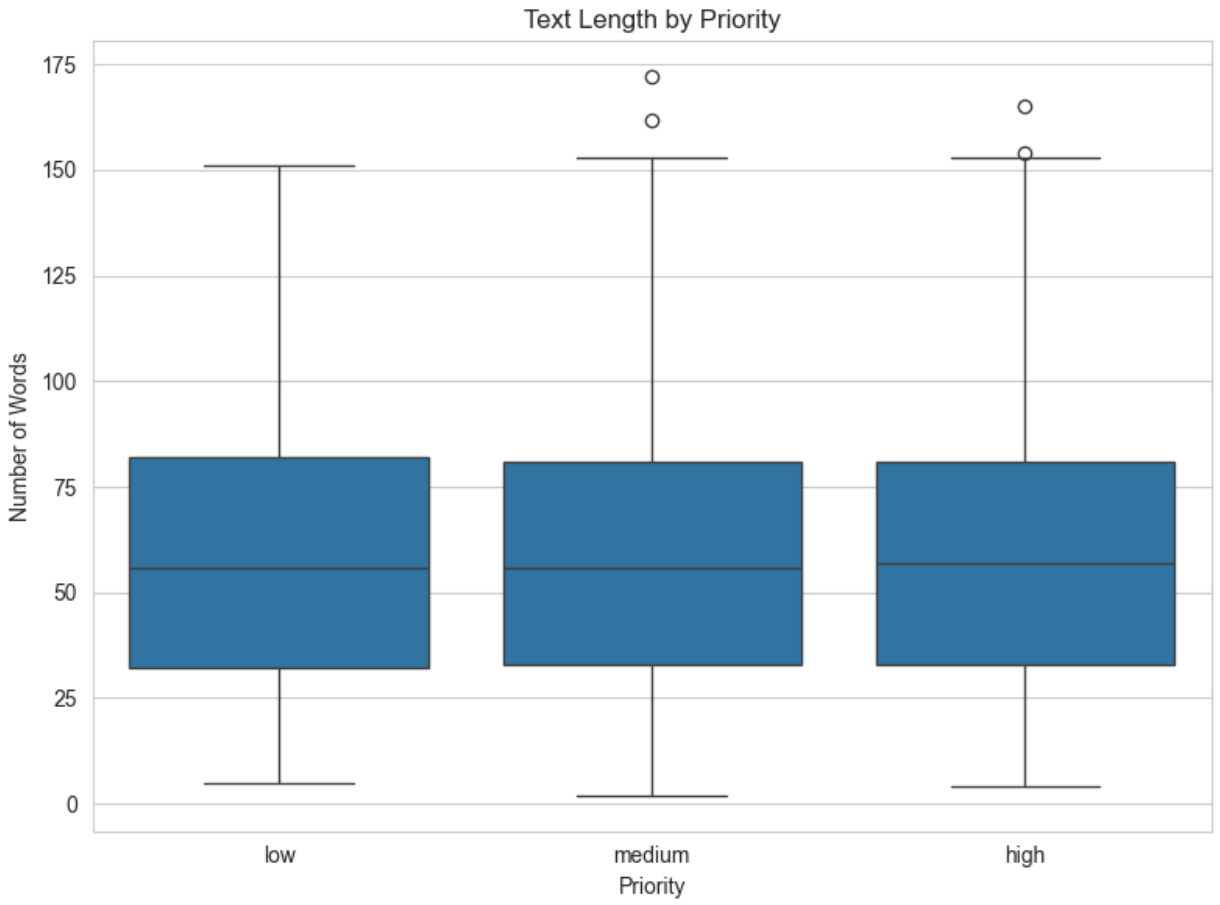
A **stratified split** based on Queue labels was used to preserve the class distribution across all subsets.

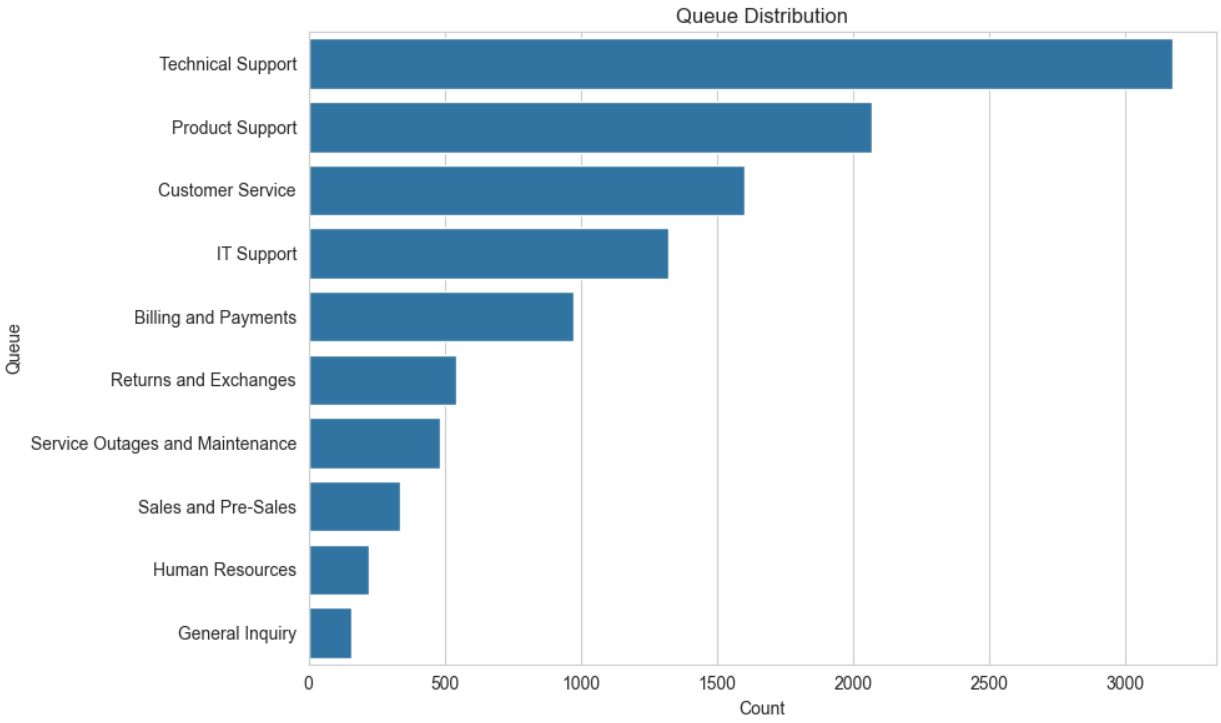
The processed EDA confirmed that the class proportions remained nearly identical after splitting.

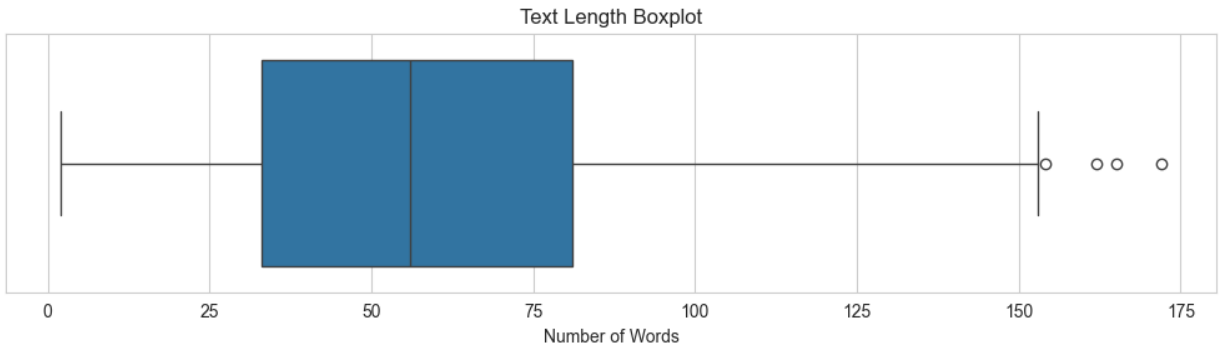
Include:

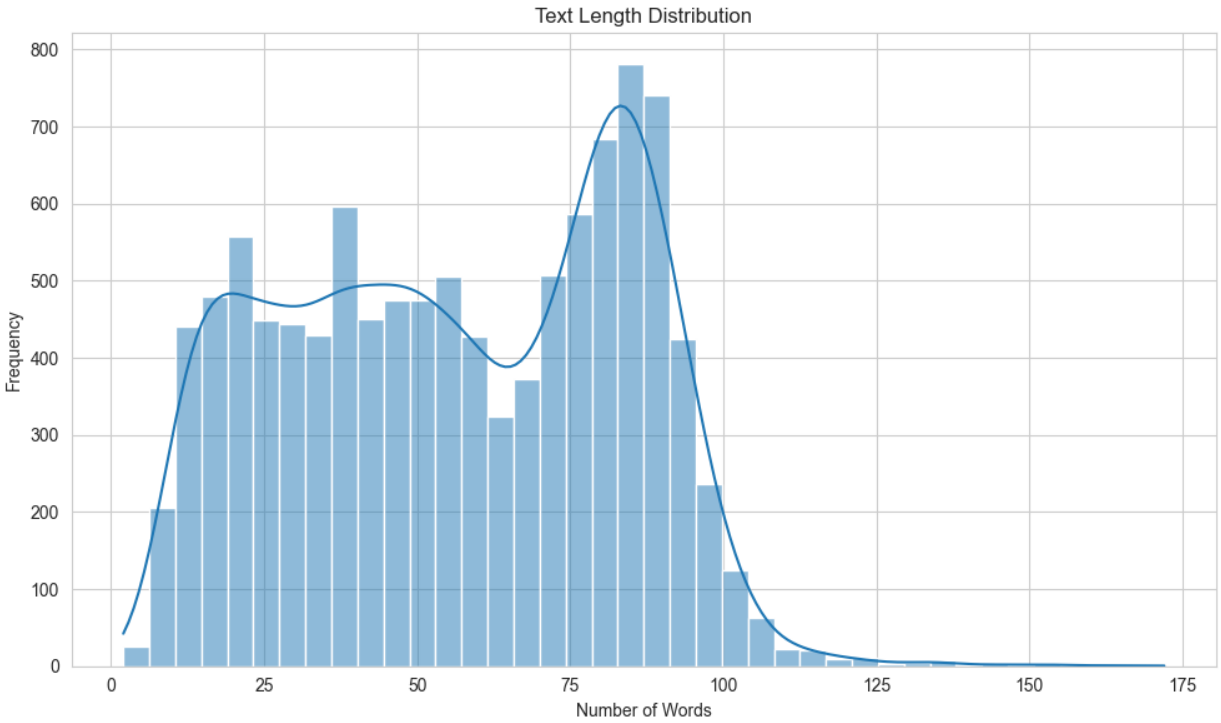
- Queue Distribution after preprocessing

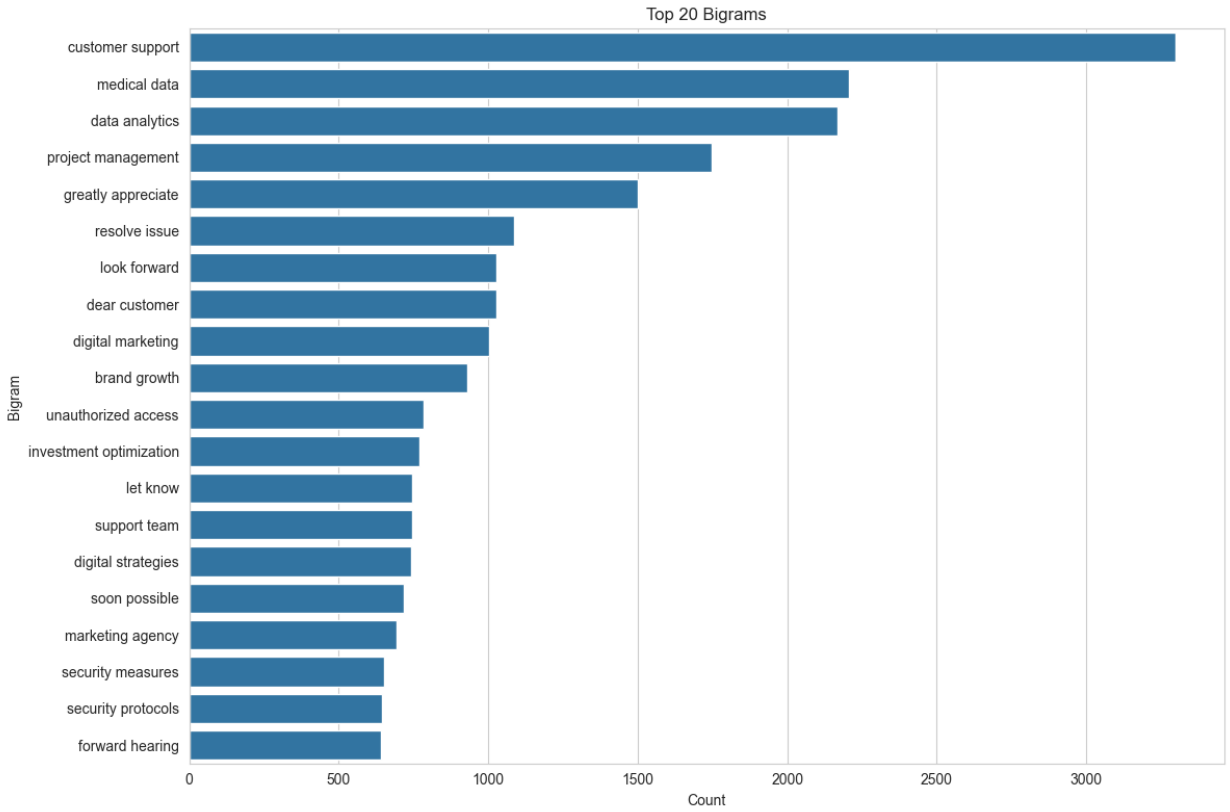
- Text Length Histogram
- Text Length Boxplot
- Text Length by Priority

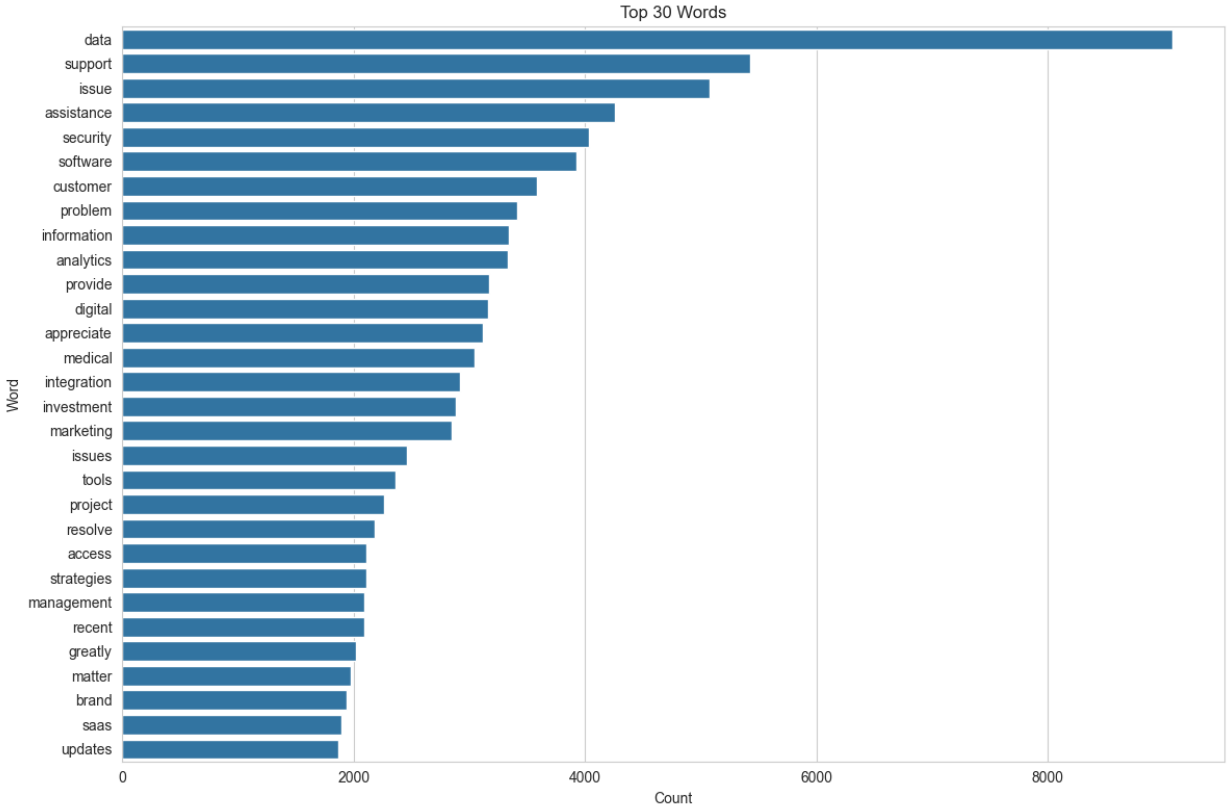

Text Length by Priority
Number of Words
175
150
125
100
75
50
25
0
low
medium
high
Priority


Queue Distribution
Queue
Technical Support
Product Support
Customer Service
IT Support
Billing and Payments
Returns and Exchanges
Service Outages and Maintenance
Sales and Pre-Sales
Human Resources
General Inquiry
0
500
1000
1500
2000
2500
3000
Count


Text Length Boxplot
0
25
50
75
100
125
150
175
Number of Words


Text Length Distribution
Frequency
800
700
600
500
400
300
200
100
0
0
25
50
75
100
125
150
175
Number of Words


Top 20 Bigrams
customer support
medical data
data analytics
project management
greatly appreciate
resolve issue
look forward
dear customer
digital marketing
brand growth
unauthorized access
investment optimization
let know
support team
digital strategies
soon possible
marketing agency
security measures
security protocols
forward hearing
Bigram
0
500
1000
1500
2000
2500
3000
Count


Top 30 Words
data
support
issue
assistance
security
software
customer
problem
information
analytics
provide
digital
appreciate
medical
integration
investment
marketing
issues
tools
project
resolve
access
strategies
management
recent
greatly
matter
brand
saas
updates
Word
0
2000
4000
6000
8000
Count

## 2.8 Feature Engineering

Two different text representations were generated.

### TF-IDF

Traditional machine learning models were trained using TF-IDF vectors.

Configuration:

- Maximum features: 30,000
- N-grams: 1–3
- Minimum document frequency: 2
- English stopword removal
- Sublinear TF scaling

Vocabulary size:

30,000

### Sentence Embeddings

For semantic retrieval and RAG, dense sentence embeddings were generated using Sentence-BERT.

Model:

**all-MiniLM-L6-v2**

Embedding dimension:

384

The generated embeddings were later used to build the FAISS vector database for Retrieval-Augmented Generation.

## 2.9 Leakage Analysis

Because semantic retrieval can artificially inflate evaluation scores if identical tickets appear in both training and testing sets, a leakage analysis was performed.

Each testing ticket was compared against all training tickets using cosine similarity.

Threshold:

0.97

Result:

- Near duplicates found: **0**
- Maximum similarity: **0.95**

This confirms that the train and test datasets are independent and that the evaluation results accurately reflect model performance.

---

# 3-Milestone 2: Model Development and Evaluation

## 3.1 Objective

The objective of this milestone was to develop and evaluate different approaches for automatic support ticket classification. Three approaches were investigated:

- Traditional Machine Learning using TF-IDF features.
- Transformer-based classification using DistilBERT.
- Retrieval-Augmented Generation (RAG) using semantic retrieval and similarity-based voting.

The three approaches were compared on the tasks of predicting the ticket **Type**, **Queue**, and **Priority** to identify the most effective solution for the customer support domain.

## 3.2 Traditional Machine Learning

As a strong baseline, traditional machine learning models were trained using the TF-IDF features generated during Milestone 1.

Three classifiers were evaluated:

- Logistic Regression
- Multinomial Naïve Bayes
- Linear Support Vector Classifier (LinearSVC)

For LinearSVC, Grid Search with 3-fold cross-validation was performed to determine the optimal regularization parameter (C). The best model for each prediction task was selected according to the weighted F1-score on the validation dataset before being evaluated on the testing dataset.

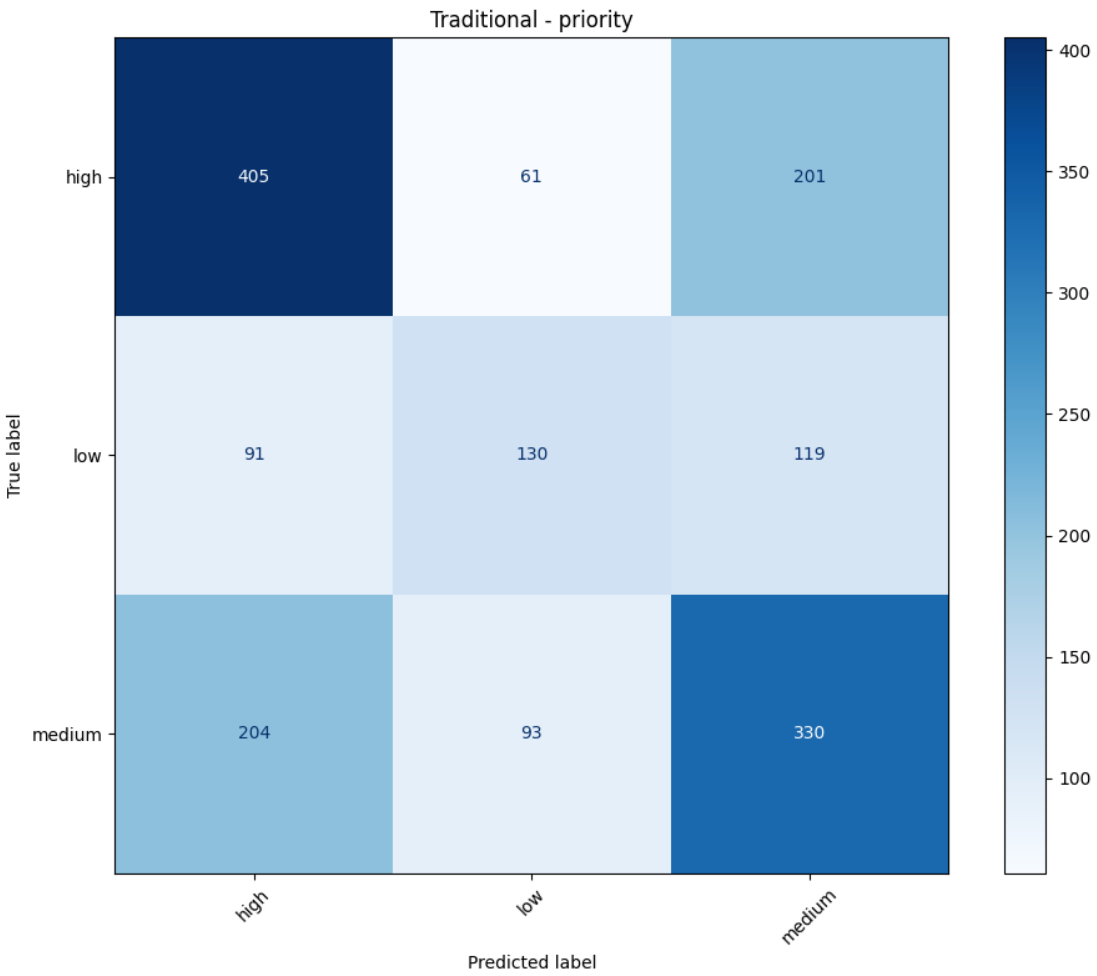
**Hyperparameters:**

| Parameter | Value |
|---|---|
| Feature Representation | TF-IDF |
| Maximum Features | 30,000 |
| Grid Search | Yes |
| Cross Validation | 3-fold |
| Evaluation Metric | Weighted F1 |

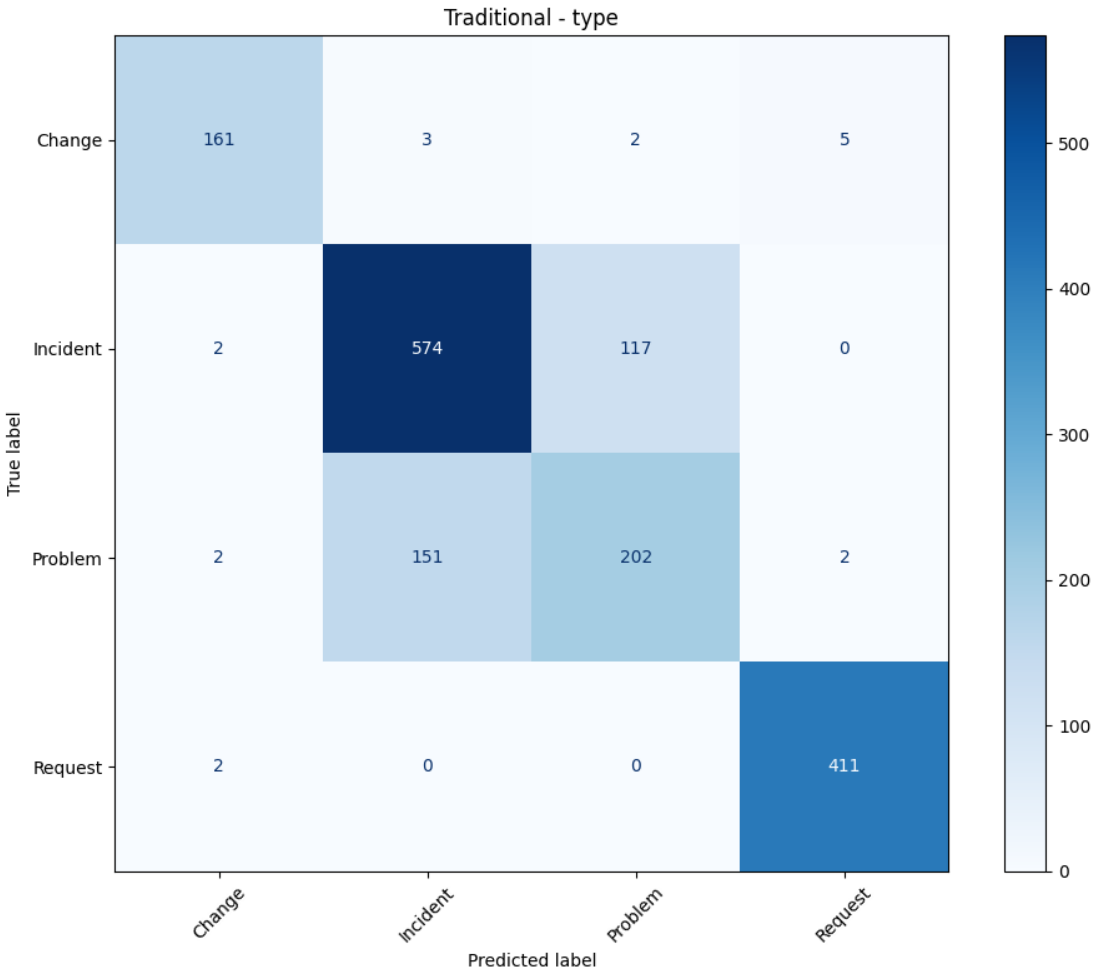
**Results**

| Task | Best Model | Accuracy | F1-score |
|---|---|---|---|
| Type | LinearSVC | **82.5%** | **82.25%** |
| Queue | LinearSVC | **47.98%** | **47.70%** |
| Priority | LinearSVC | **52.94%** | **52.69%** |

LinearSVC achieved the best performance across all three prediction tasks and was selected as the traditional baseline for comparison.


Traditional - priority
high
low
medium
True label
405
61
201
91
130
119
204
93
330
high
low
medium
Predicted label
400
350
300
250
200
150
100


Traditional - queue
Billing and Payments
Customer Service
General Inquiry
Human Resources
IT Support
Product Support
Returns and Exchanges
Sales and Pre-Sales
Service Outages and Maintenance
Technical Support
True label
Predicted label
110 11 0 0 4 9 2 1 0 9
11 97 1 6 22 41 6 2 4 50
0 4 9 0 0 4 1 0 0 6
0 4 0 18 0 6 2 0 0 3
4 18 0 1 64 30 5 3 7 67
7 45 3 3 26 134 6 4 4 78
1 12 0 0 5 13 27 5 1 18
3 9 1 0 1 11 4 11 0 10
2 3 0 1 6 4 0 2 50 5
5 46 2 3 42 84 9 8 14 264
250
200
150
100
50
0




## 3.3 DistilBERT Classification

To evaluate the effectiveness of transformer-based models, DistilBERT was fine-tuned separately for each classification task.

The dataset was tokenized using the DistilBERT tokenizer, and a separate classification model was trained for predicting ticket Type, Queue, and Priority.

Training employed:

- Early stopping
- Adam optimizer
- Learning rate scheduling
- Weight decay
- Validation after every epoch

These techniques were used to improve convergence and reduce overfitting on the relatively small training dataset.

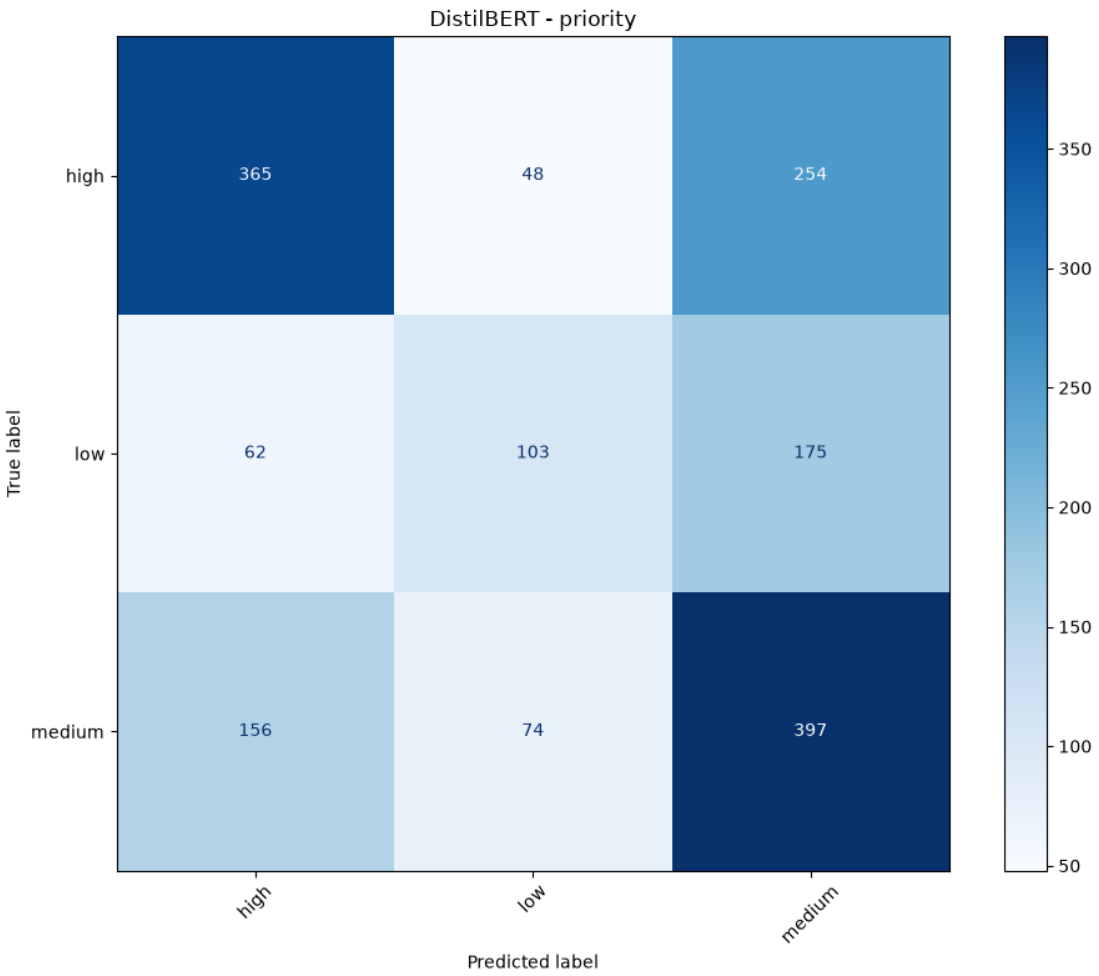
**Training Configuration**

| **Parameter** | **Value** |
|---|---|
| Model | DistilBERT-base-uncased |
| Maximum Length | 128 |
| Batch Size | 8 |
| Learning Rate | $2\times10^{-5}$ |
| Maximum Epochs | 10 |
| Early Stopping | Patience = 2 |

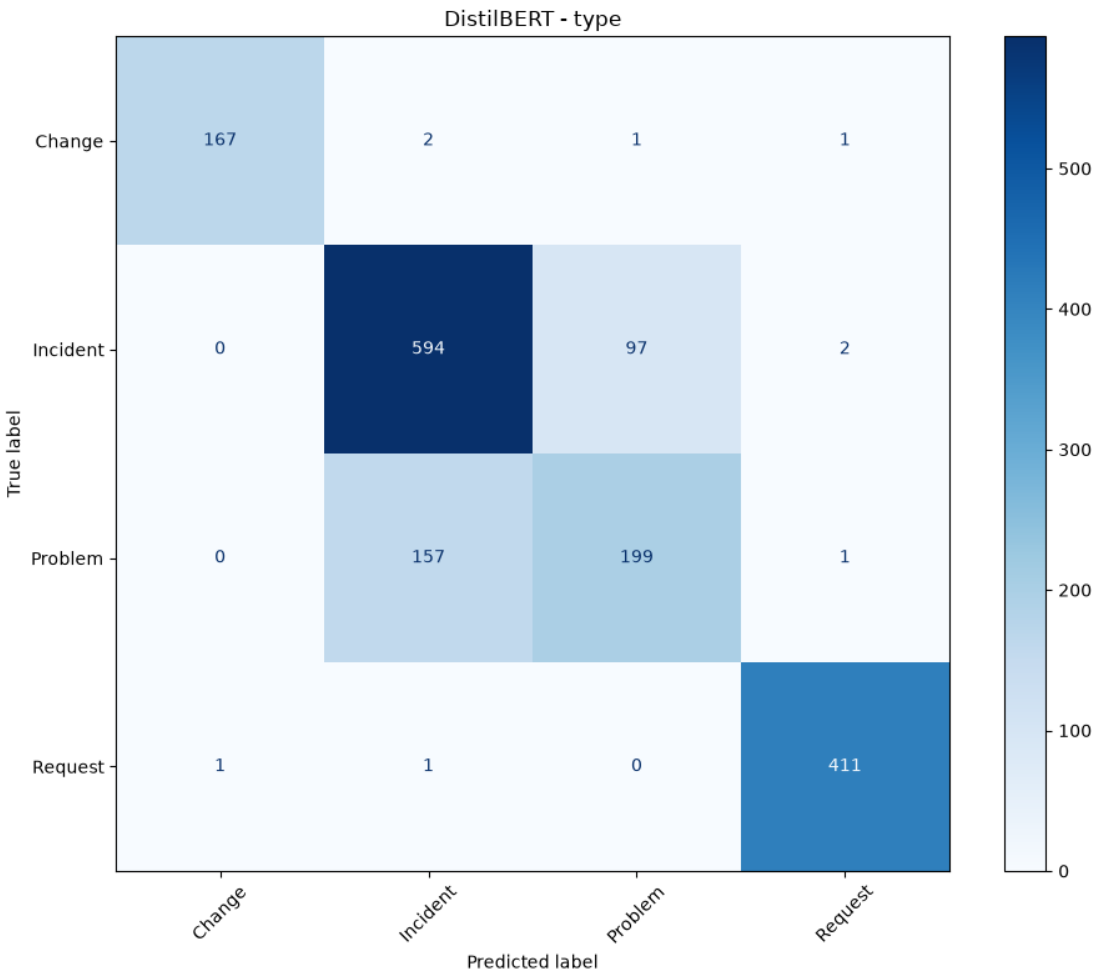
**Results:**

| **Task** | **Accuracy** | **F1-score** |
|---|---|---|
| Type | **83.90%** | **83.52%** |
| Queue | **46.14%** | **44.86%** |
| Priority | **52.94%** | **52.39%** |

DistilBERT achieved the highest performance for **ticket type prediction**, demonstrating the benefit of contextual language understanding for coarse-grained classification tasks.


DistilBERT - priority
high
low
medium
True label
365
48
254
62
103
175
156
74
397
high
low
medium
Predicted label
350
300
250
200
150
100
50


DistilBERT - queue
Billing and Payments
Customer Service
General Inquiry
Human Resources
IT Support
Product Support
Returns and Exchanges
Sales and Pre-Sales
Service Outages and Maintenance
Technical Support
True label
Predicted label
107 9 0 0 4 16 0 0 0 10
9 80 0 1 14 62 13 6 3 52
3 1 2 2 2 10 0 2 0 2
1 3 0 10 0 14 0 0 1 4
6 9 0 0 50 29 9 1 4 91
11 28 0 2 15 140 11 3 3 97
4 11 0 0 4 23 19 1 0 20
4 11 1 0 2 17 2 4 1 8
2 4 0 0 7 4 0 0 46 10
8 33 0 3 30 88 11 2 6 296
250
200
150
100
50
0




## 3.4 Retrieval-Augmented Generation (RAG)

Unlike the previous approaches, RAG was not trained as a supervised classifier.

Instead, it consisted of three stages:

1. Encode incoming tickets using Sentence-BERT.
2. Retrieve the most similar historical tickets using a FAISS vector index.
3. Predict ticket labels using similarity-weighted voting while using retrieved examples as context for response generation.

The retrieval system was built using:

- Sentence-BERT embeddings
- FAISS IndexFlatIP
- Cosine similarity
- Similarity threshold filtering
- Weighted majority voting

For response generation, retrieved answers were provided as context to the **Qwen2.5-1.5B-Instruct** language model.

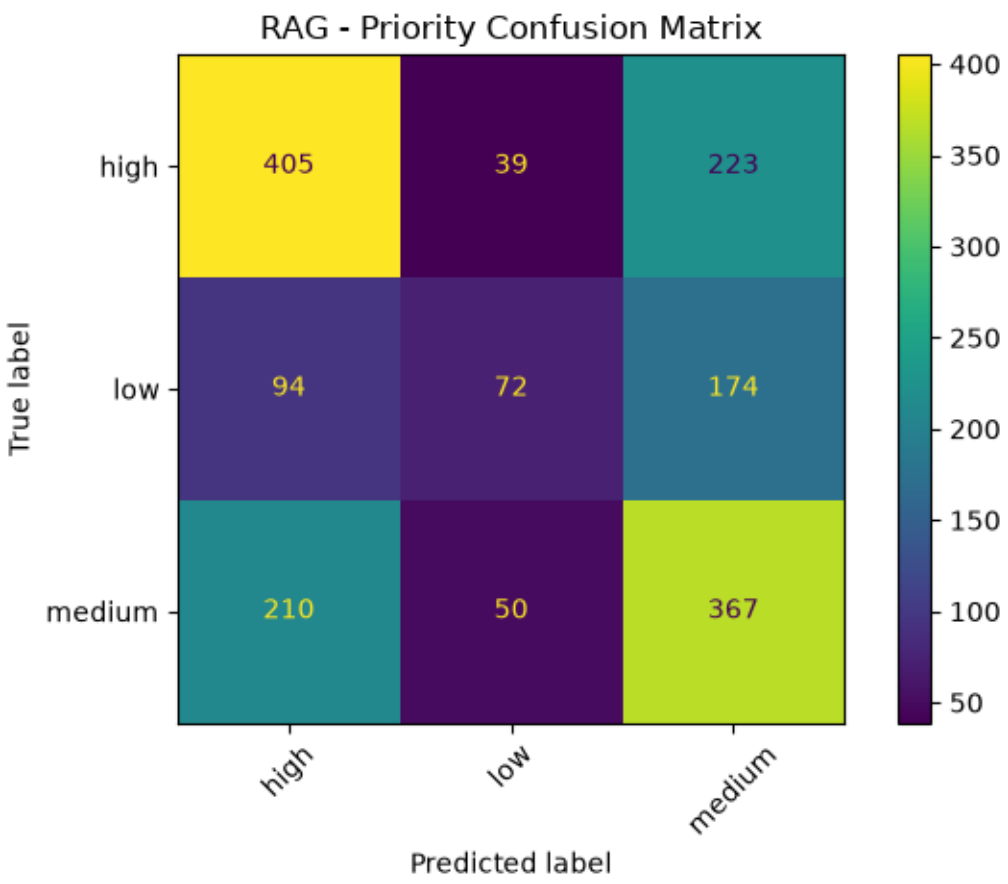
**Classification Results**

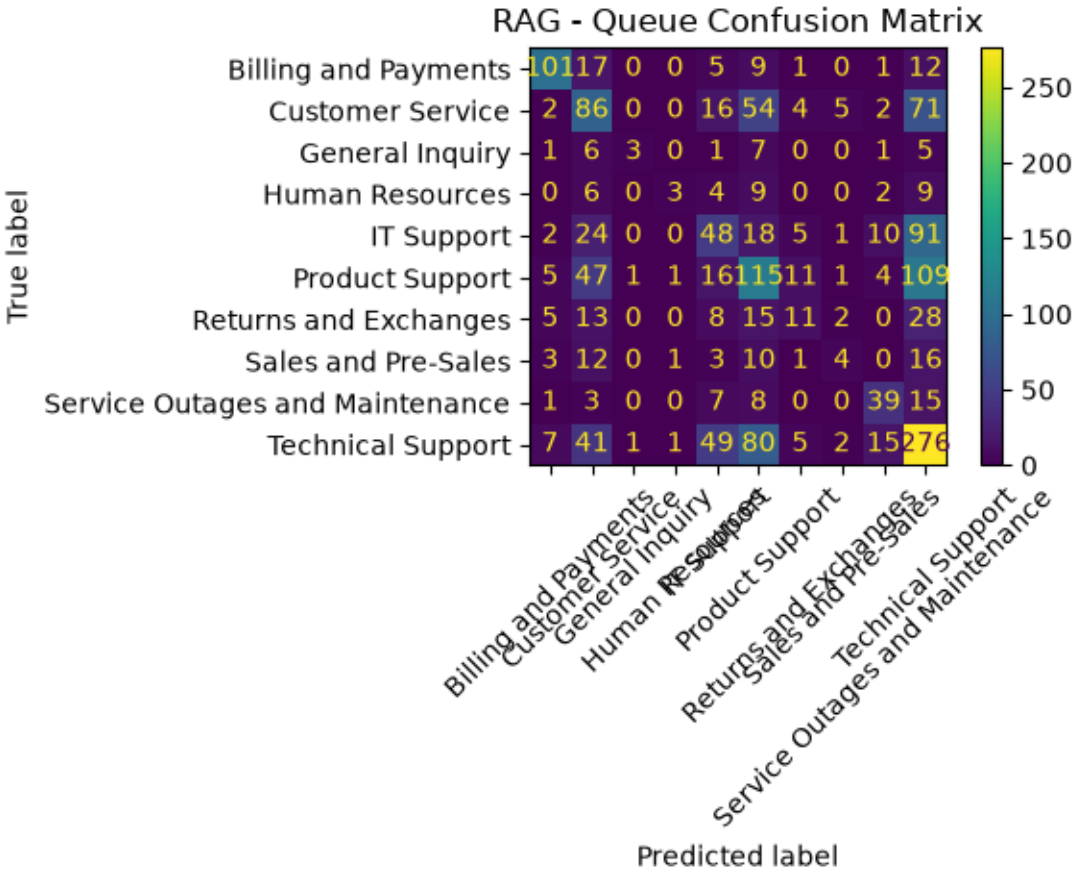
| Task | Accuracy |
|---|---|
| Type | 79.31% |
| Queue | 41.98% |
| Priority | 51.65% |

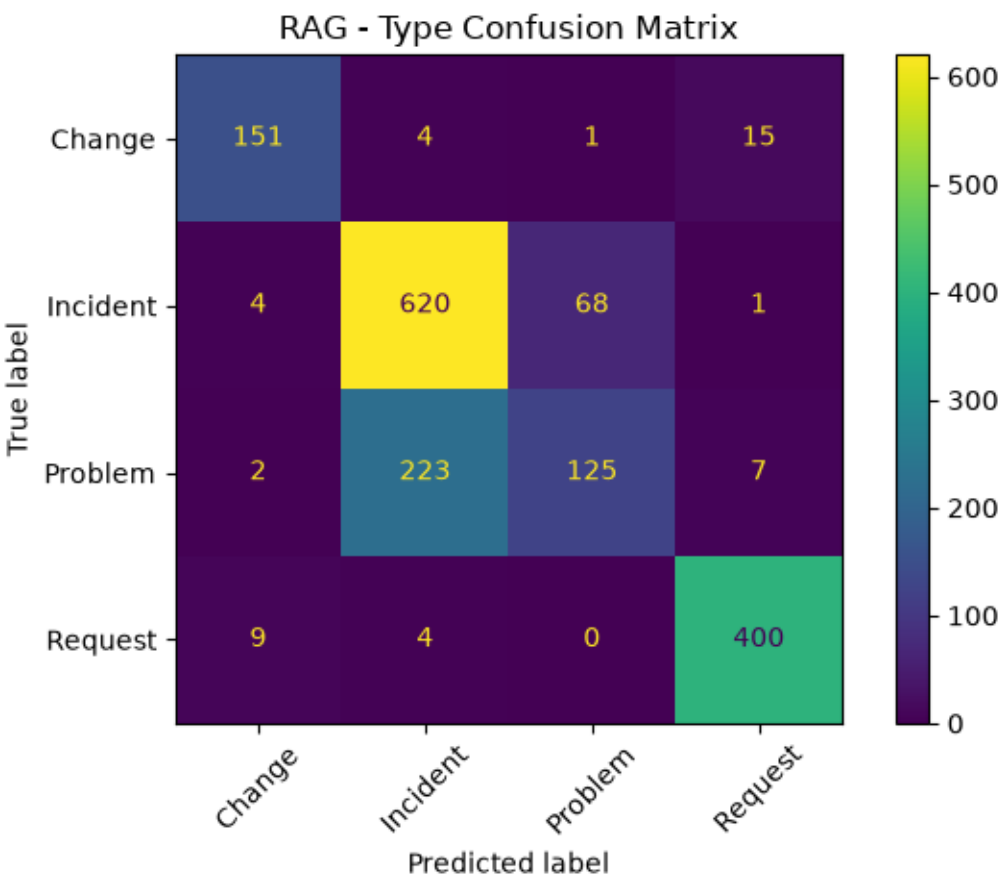
**Average BLEU Score**

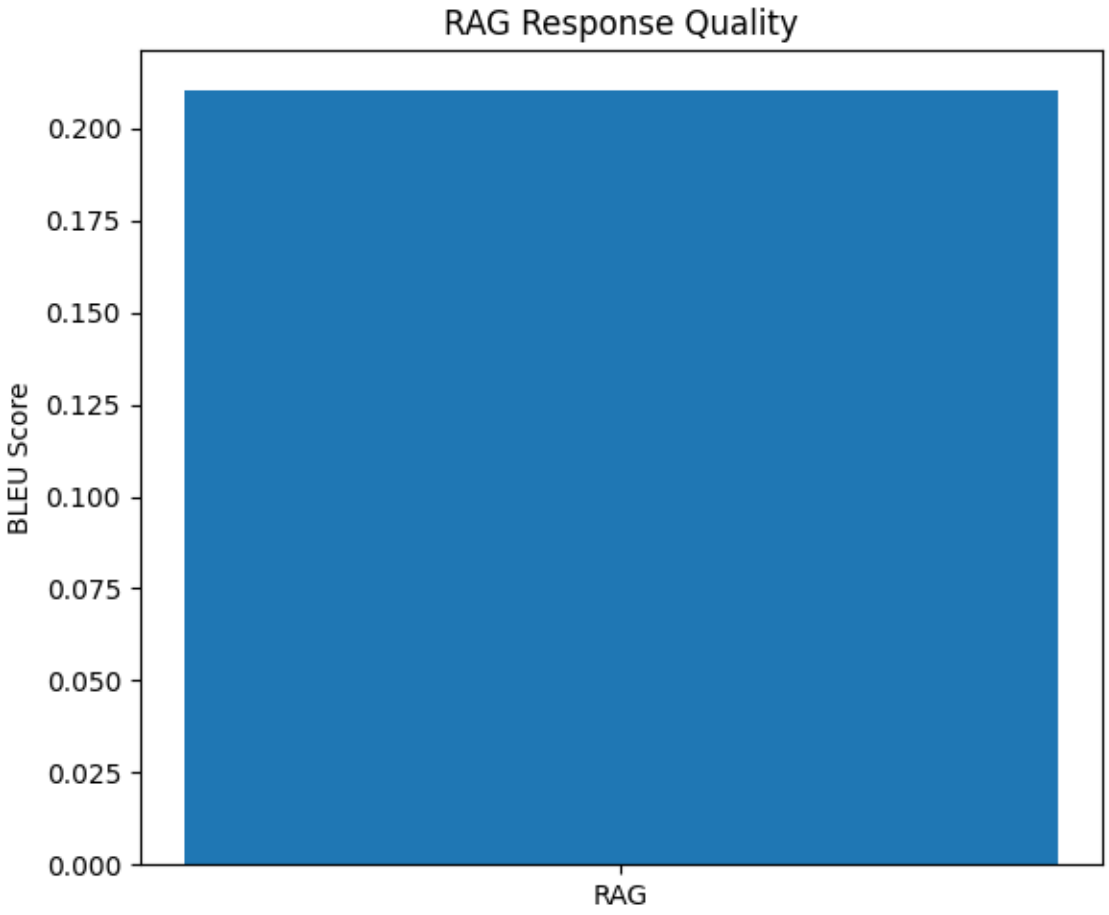
0.2106

Although classification performance was lower than the supervised models, RAG successfully generated context-aware responses by incorporating similar historical support cases

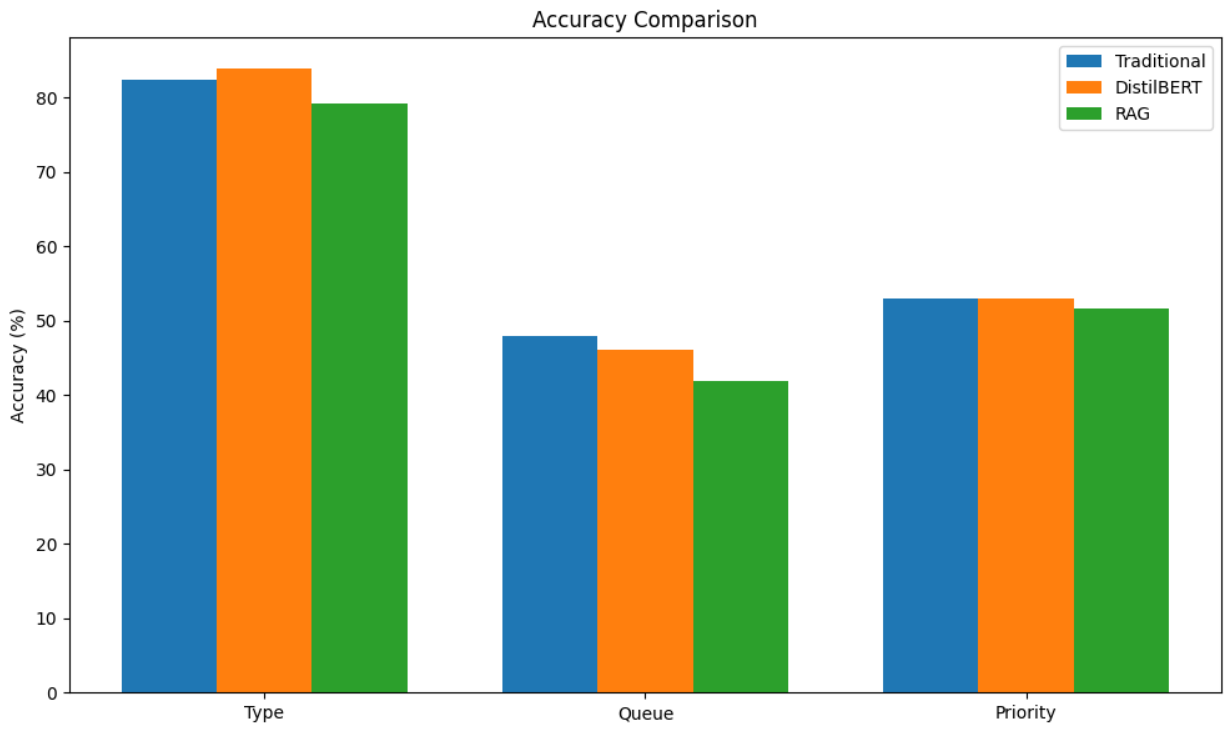

RAG - Priority Confusion Matrix
high
low
medium
True label
405
39
223
94
72
174
210
50
367
high
low
medium
Predicted label
400
350
300
250
200
150
100
50

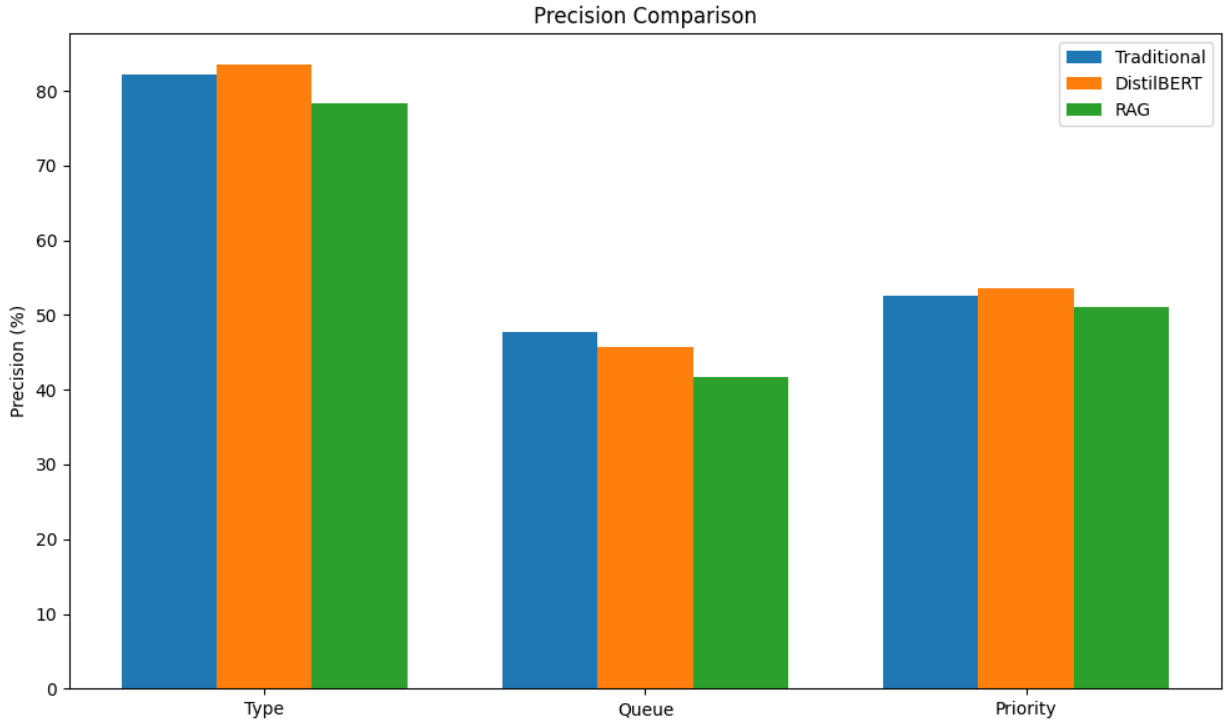

RAG - Queue Confusion Matrix
True label
Billing and Payments
Customer Service
General Inquiry
Human Resources
IT Support
Product Support
Returns and Exchanges
Sales and Pre-Sales
Service Outages and Maintenance
Technical Support
10 17 0 0 5 9 1 0 1 12
2 86 0 0 16 54 4 5 2 71
1 6 3 0 1 7 0 0 1 5
0 6 0 3 4 9 0 0 2 9
2 24 0 0 48 18 5 1 10 91
5 47 1 1 16 115 11 1 4 109
5 13 0 0 8 15 11 2 0 28
3 12 0 1 3 10 1 4 0 16
1 3 0 0 7 8 0 0 39 15
7 41 1 1 49 80 5 2 15 276
250
200
150
100
50
0
Billing and Payments
Customer Service
General Inquiry
Product Support
Returns and Exchanges
Sales and Pre-Sales
Technical Support
Service Outages and Maintenance
Predicted label

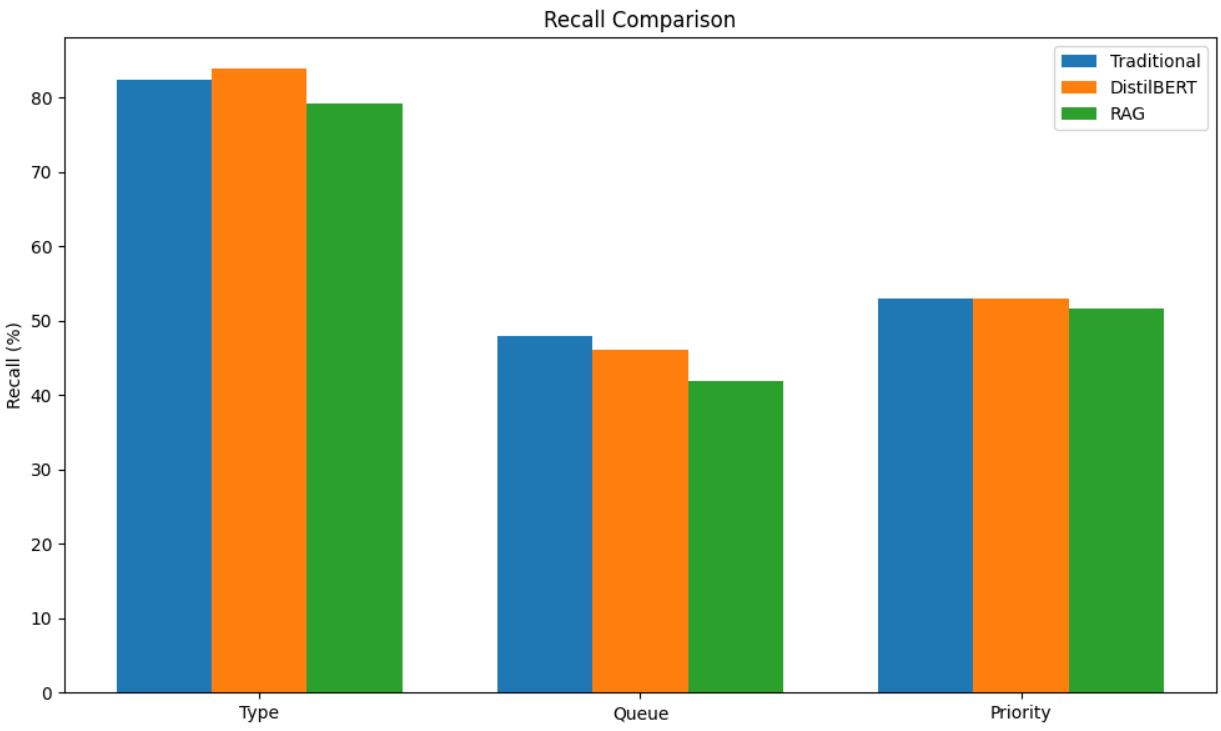

RAG - Type Confusion Matrix
Change
Incident
Problem
Request
True label
151
4
1
15
4
620
68
1
2
223
125
7
9
4
0
400
Change
Incident
Problem
Request
Predicted label
600
500
400
300
200
100
0

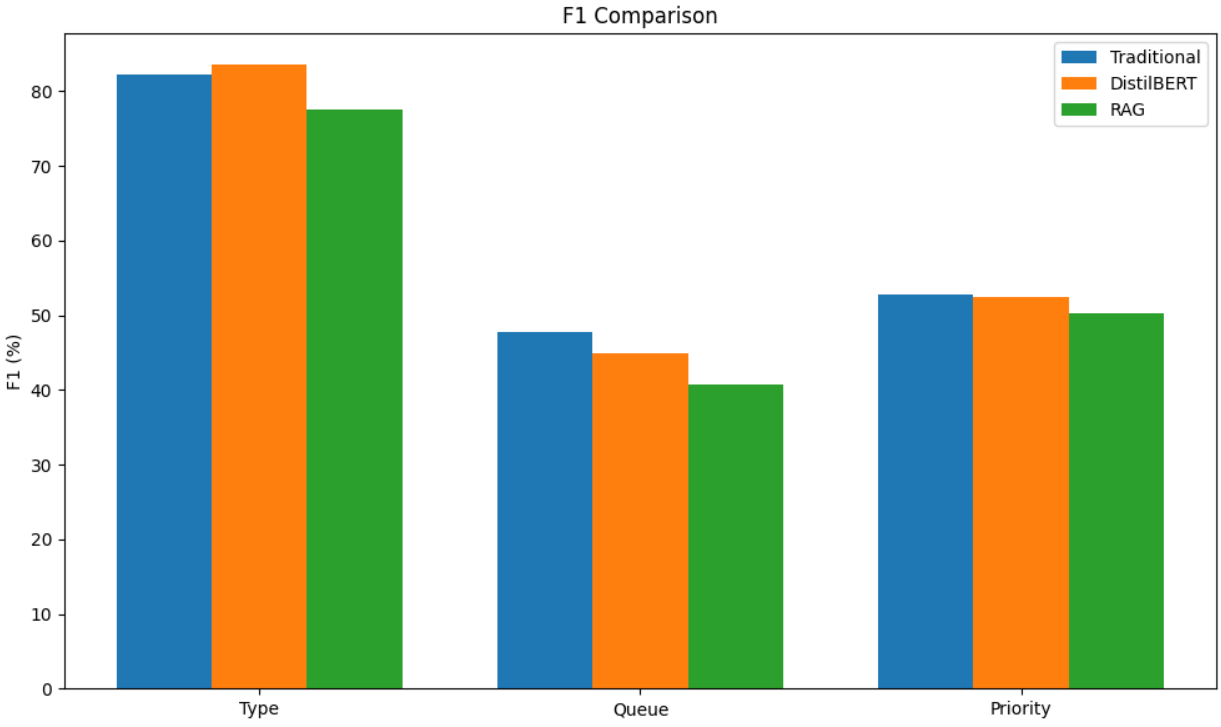


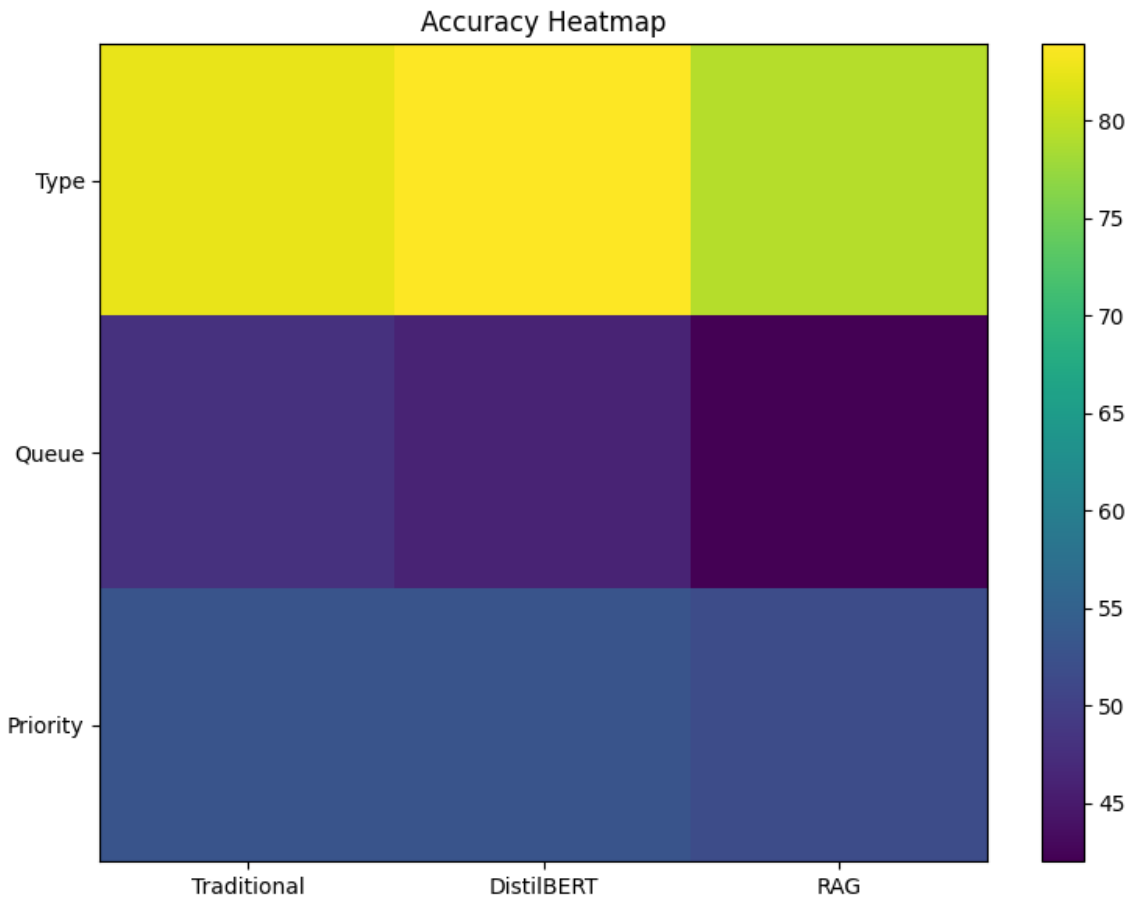

**Model Comparison**


Accuracy Comparison
Traditional
DistilBERT
RAG
Accuracy (%)
0
10
20
30
40
50
60
70
80
Type
Queue
Priority


Precision Comparison
Traditional
DistilBERT
RAG
Precision (%)
0
10
20
30
40
50
60
70
80
Type
Queue
Priority


Recall Comparison
Traditional
DistilBERT
RAG
Recall (%)
0
10
20
30
40
50
60
70
80
Type
Queue
Priority


F1 Comparison
Traditional
DistilBERT
RAG
F1 (%)
0
10
20
30
40
50
60
70
80
Type
Queue
Priority




Overall observations include:

- Traditional LinearSVC achieved the highest performance on Queue and Priority prediction.
- DistilBERT achieved the highest accuracy for Type prediction.
- RAG obtained lower classification accuracy but enabled high-quality response generation, which traditional classifiers cannot provide.

## Effect of Data Leakage

Initial experiments showed that the RAG model significantly outperformed the other approaches. However, further investigation revealed that approximately 31% of testing tickets had nearly identical counterparts within the training dataset.

Since RAG retrieves the most similar historical tickets, these duplicates effectively allowed the system to retrieve the correct answer directly, leading to unrealistically high classification performance.

After introducing semantic duplicate removal before dataset splitting, this leakage was eliminated, resulting in a fair comparison between all approaches.

### Why Traditional Machine Learning Performed Best

Following duplicate removal, LinearSVC combined with TF-IDF achieved the highest performance for Queue and Priority prediction.

This outcome can be explained by the characteristics of the dataset.

Many ticket categories are distinguished primarily by explicit keywords such as **refund**, **billing**, **login**, **server**, and **outage**.

TF-IDF emphasizes these discriminative terms directly, allowing LinearSVC to learn effective decision boundaries with relatively limited training data.

---

### DistilBERT Performance

DistilBERT achieved the highest accuracy for predicting ticket Type because this task benefits more from contextual semantic understanding.

However, transformer models require larger datasets to fully exploit their representational power. After removing near-duplicate tickets, only approximately **7,600** training samples remained, limiting the potential advantages of fine-tuning.

---

### Role of RAG

The Retrieval-Augmented Generation pipeline achieved the lowest classification accuracy because similarity-based voting over a small number of retrieved examples is fundamentally different from supervised learning.

Nevertheless, RAG serves a different purpose within the overall system.

Its primary contribution is generating context-aware customer responses by incorporating similar historical support cases, rather than maximizing classification accuracy.

Consequently, RAG remains an essential component of the complete support ticket system despite its lower classification metrics.

---

# 4- Milestone 3 - SYSTEM INTEGRATION & DEPLOYMENT

## 4.1 Objective

The objective of this milestone was to transform the trained AI models into a complete intelligent support ticket system.

Instead of evaluating models independently, this phase focused on integrating all project components into a production-style application that allows users to submit support tickets, receive AI-generated responses, and enables administrators to monitor and manage the support workflow.

The integrated system combines:

- Azure Machine Learning deployment
- Azure AI Search retrieval
- Retrieval-Augmented Generation (RAG)
- FastAPI backend services
- SQLite database
- Authentication and authorization
- Client and administrator web dashboards
- Monitoring and feedback collection

## 4.2 System Architecture

The complete workflow consists of the following stages:

1. User submits a support ticket through the client dashboard.
2. The backend authenticates the user using JWT.
3. The ticket is sent to the deployed Azure ML endpoint for classification.
4. Azure AI Search retrieves similar historical tickets.
5. The Qwen language model generates an AI response using retrieved context.
6. Predictions and responses are stored in the database.
7. The response is returned to the client.
8. Clients can provide feedback or reopen unresolved tickets.
9. Administrators monitor and resolve reopened or failed cases.

This architecture separates AI inference, retrieval, storage, and presentation into independent components, improving scalability and maintainability.

## 4.3 Backend Development

The backend was implemented using **FastAPI**, providing RESTful APIs for communication between the frontend, database, and AI services.

Major backend modules include:

| **Module** | **Purpose** |
|---|---|
| Authentication | User login/signup using JWT |
| Ticket API | Ticket submission and retrieval |
| Azure ML Service | Ticket classification |

| Module | Purpose |
|---|---|
| Azure AI Search | Similar ticket retrieval |
| Qwen Service | AI response generation |
| Database Layer | Store users, tickets, responses, logs and feedback |
| Admin API | Ticket management and monitoring |

The backend coordinates communication between all AI components while exposing simple endpoints to the frontend.

## 4.4 Azure Machine Learning Deployment

The trained classification model was deployed as an online Azure Machine Learning endpoint.

The deployment exposes a REST API that accepts ticket text and returns predicted:

- Queue
- Type
- Priority

Deploying the model as a cloud service separates inference from the application, allowing the backend to request predictions without embedding the model directly into the server.

Advantages include:

- Scalable inference
- Cloud deployment
- Independent model updates
- REST API integration

## 4.5 Azure AI Search Integration

Semantic retrieval was implemented using Azure AI Search.

Instead of searching raw text, support tickets are stored as vector embeddings, allowing the retrieval system to locate semantically similar historical tickets.

The retrieval process consists of:

1. Encode incoming ticket.
2. Perform vector similarity search.
3. Retrieve Top-K similar tickets.
4. Return ticket metadata and previous resolutions.

These retrieved examples are later used by the language model during response generation.

## 4.6 Retrieval-Augmented Generation

After similar tickets are retrieved, the system generates an intelligent customer response using **Qwen2.5-7B-Instruct**.

The generation pipeline performs the following operations:

- Predict ticket labels using similarity-weighted voting.
- Remove placeholders from retrieved answers.
- Construct contextual prompts.
- Generate a professional customer support response.

Unlike simple template-based replies, the generated responses are context-aware and adapt to the user's issue using retrieved historical support cases

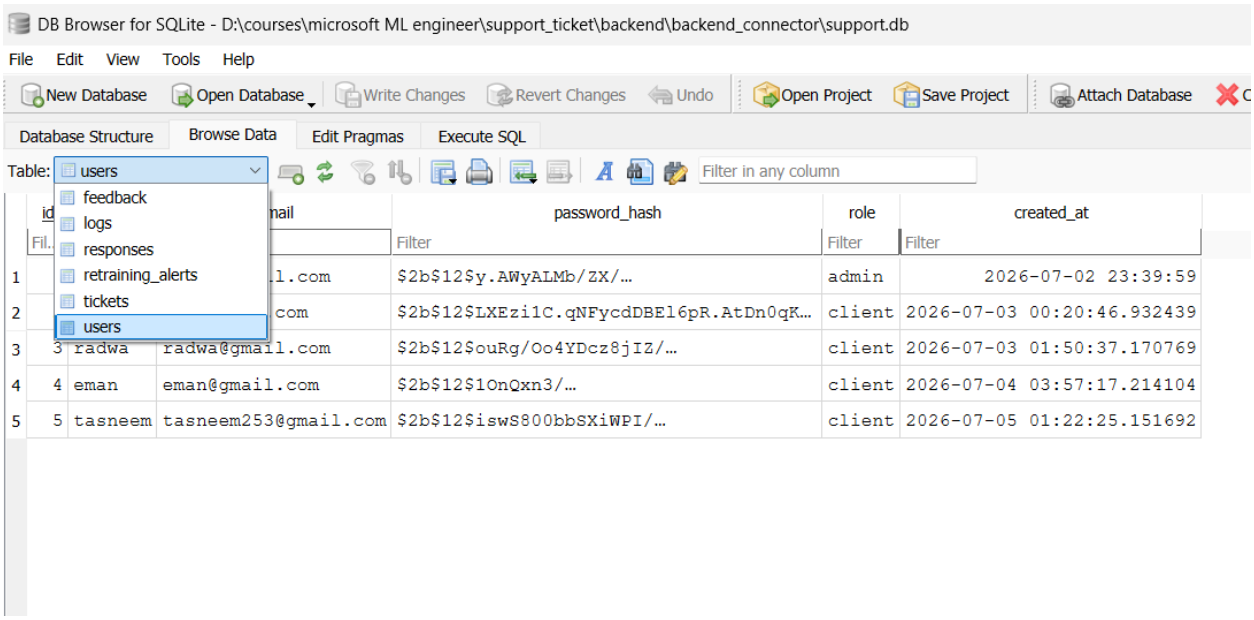
## 4.7 Database Design

SQLite was selected for data persistence due to its simplicity and suitability

The database stores:

- Users
- Tickets
- AI Responses
- Feedback
- Activity Logs

Relationships were implemented between users, tickets, and responses to support authentication, ticket history, and monitoring.

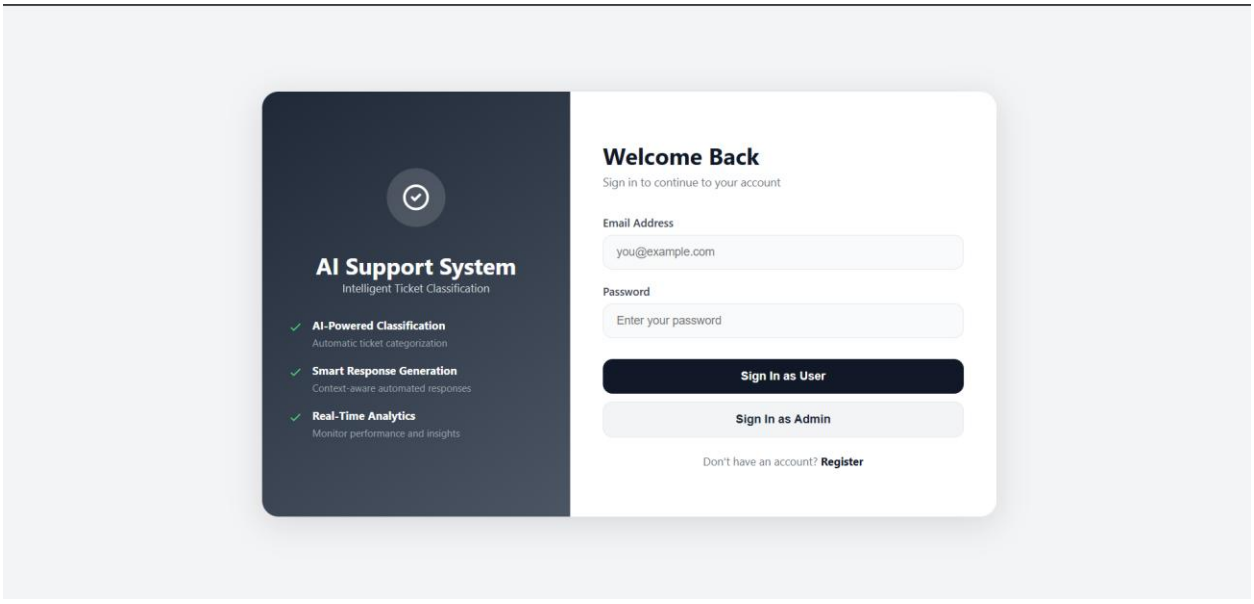



## 4.8 Authentication

The application supports secure authentication using JWT tokens.

Features include:

- User registration
- User login
- Admin login
- Password hashing
- Role-based authorization



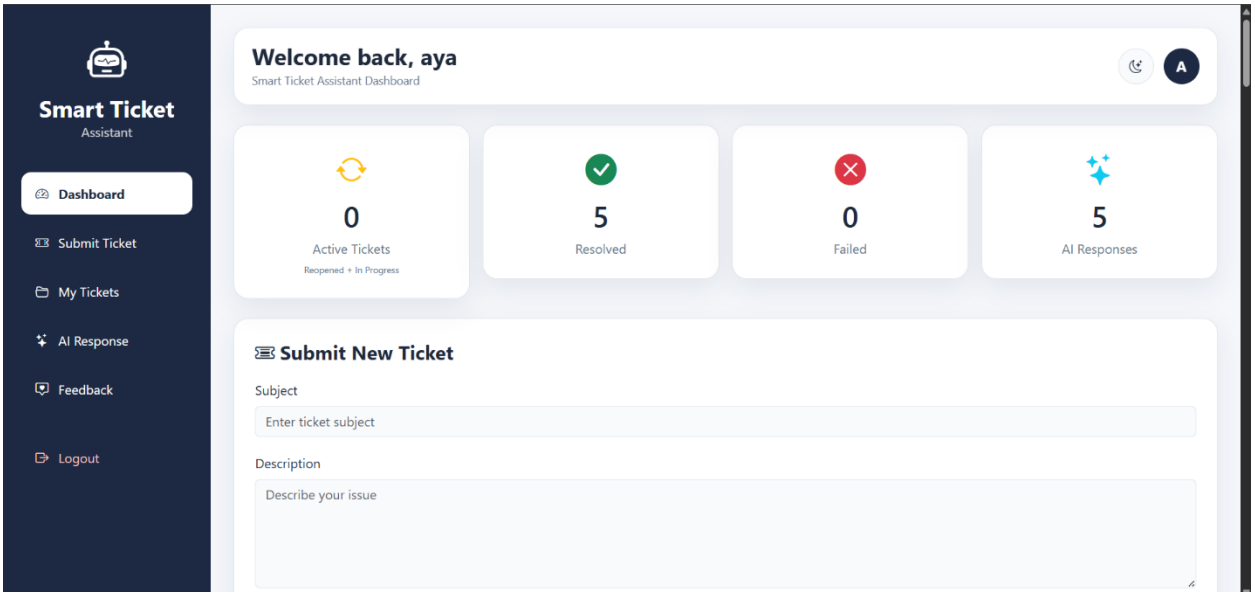
Only administrators can access the administrative dashboard, while regular users access the client dashboard.

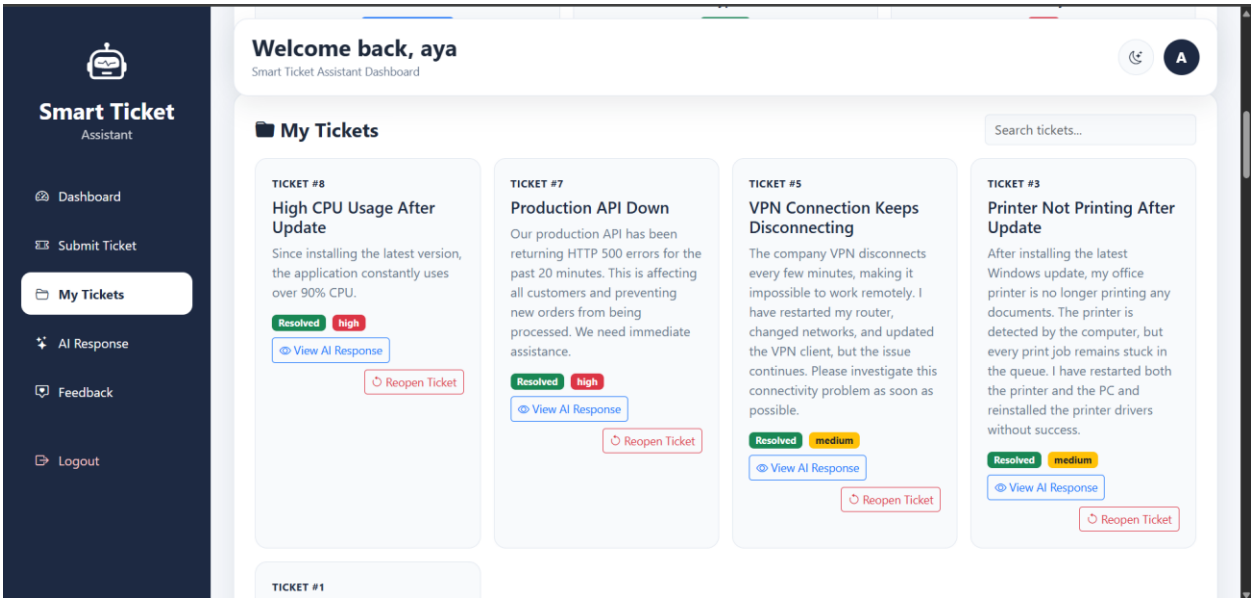
## 4.9 Client Dashboard

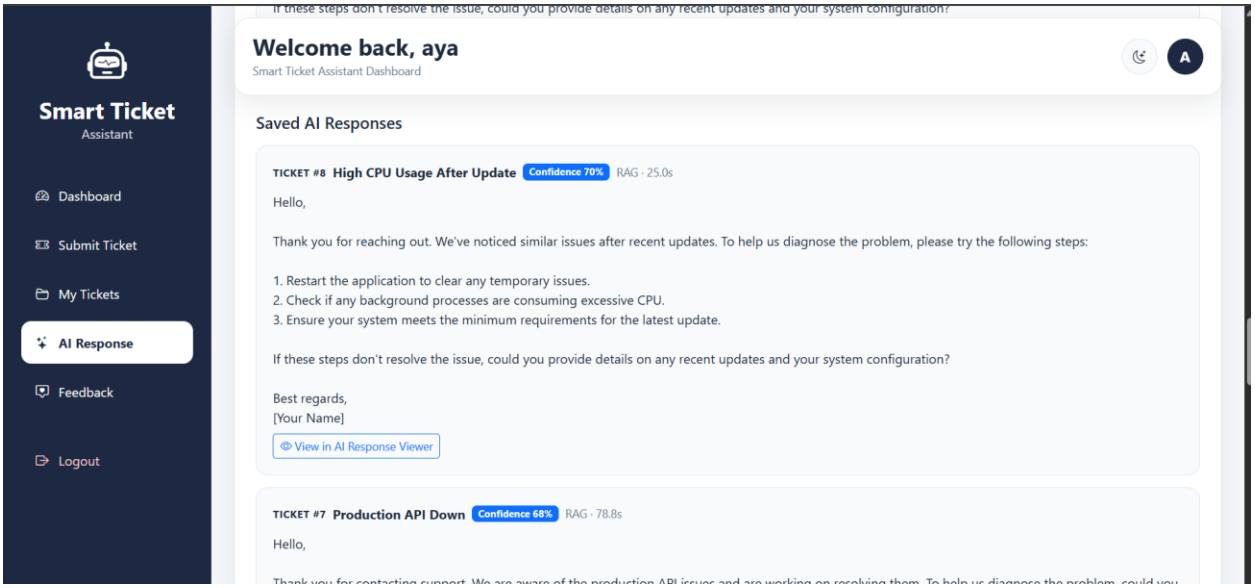
The client dashboard allows users to:

- Submit new tickets
- View AI classification results
- Receive generated AI responses
- View previous tickets
- Reopen resolved tickets
- Submit feedback

The dashboard provides a simple interface for interacting with the intelligent support system while maintaining ticket history and AI-generated responses.






## 4.10 Administrator Dashboard

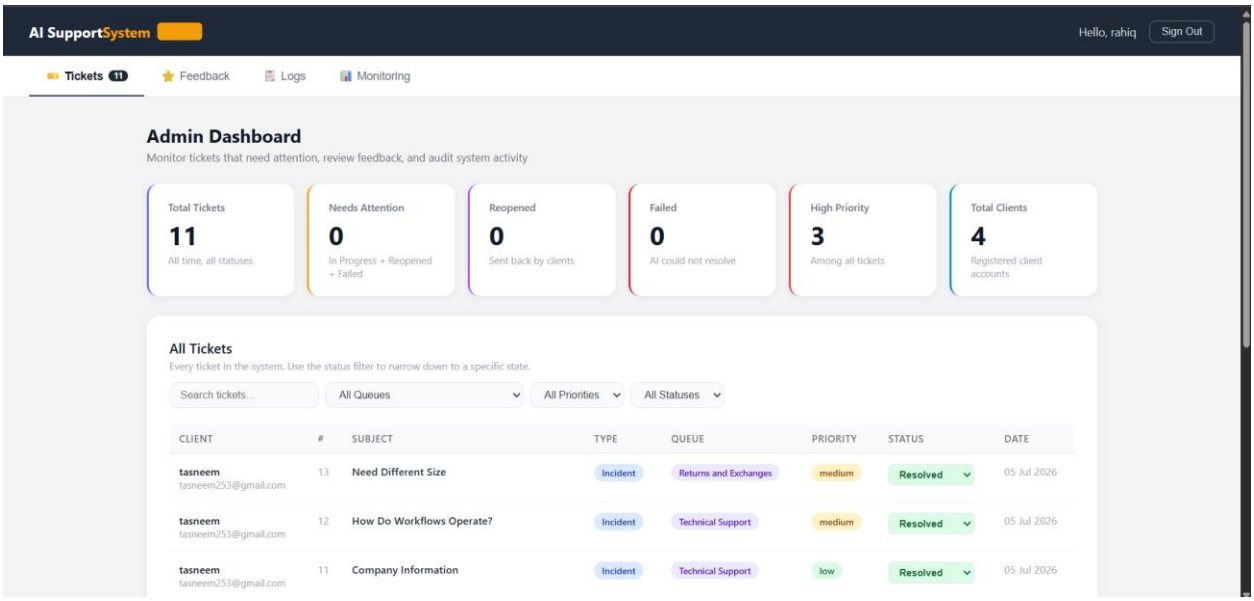
The administrator dashboard provides complete system management.

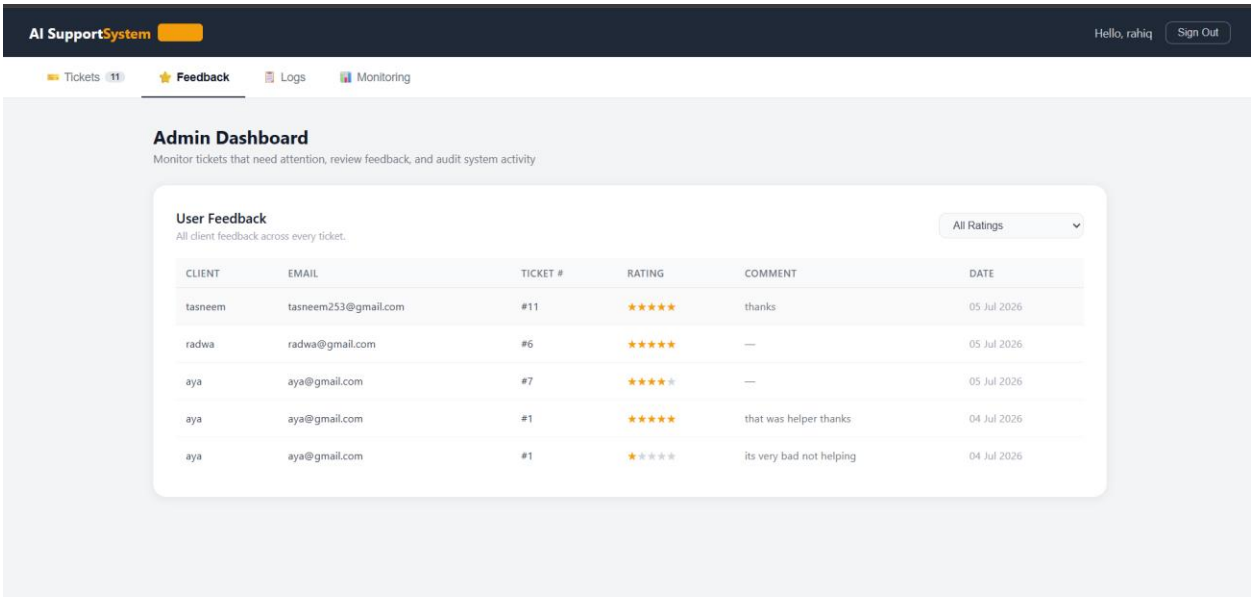
Main features include:

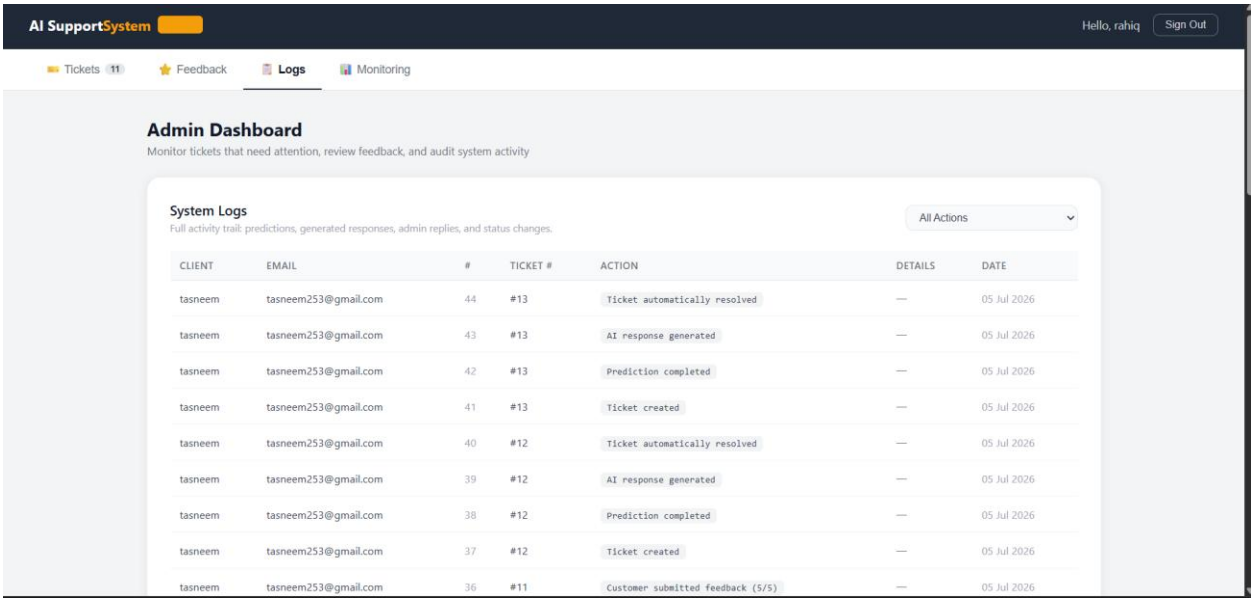
- View all tickets
- Search and filter tickets
- Update ticket status
- Review reopened cases
- Review failed AI cases
- Send manual responses

- View user feedback
- Monitor system activity logs
- Monitor system KPIs
- View retraining alerts

These tools enable human oversight while allowing AI to handle routine requests automatically.






---

# 5.milestone 4 – MLOps

## 5.1 Objective

The objective of this milestone was to introduce MLOps practices that enable continuous monitoring of the deployed AI system. Instead of treating the trained models as static components, the system monitors their performance after deployment, tracks experiments, and automatically determines when retraining is recommended.

The MLOps pipeline consists of:

- MLflow experiment tracking
- Performance monitoring
- Automated retraining checks
- Retraining alert management

## 5.2 MLflow Experiment Tracking

To ensure reproducibility and simplify model comparison, **MLflow** was integrated into the training pipeline.

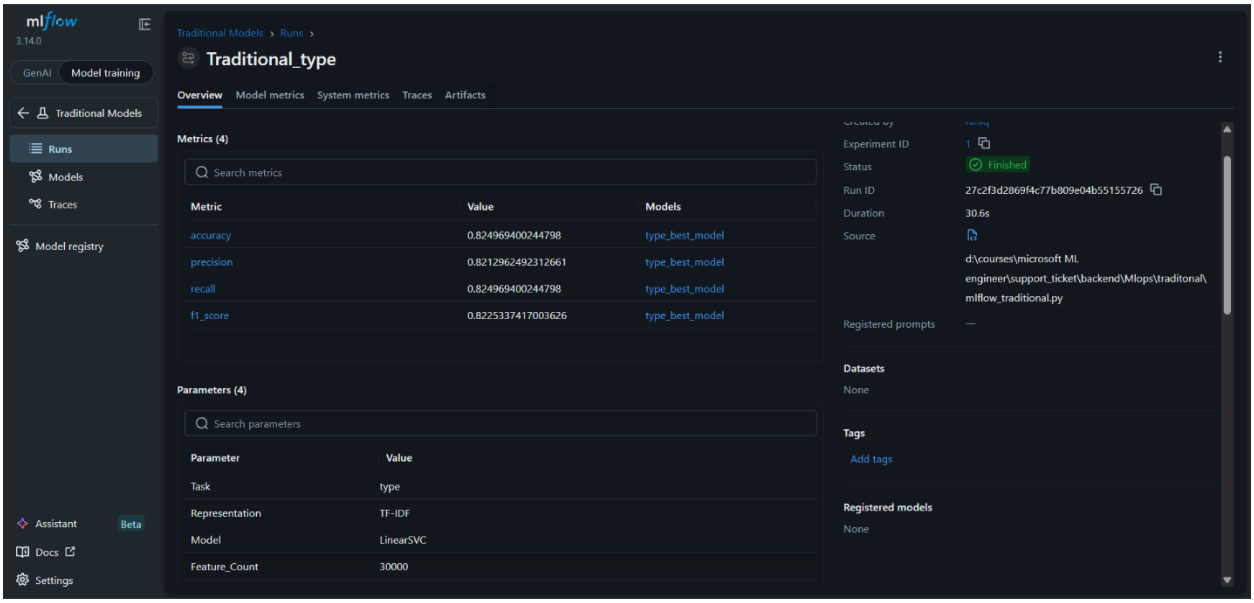
A separate MLflow experiment was created for each classification approach:

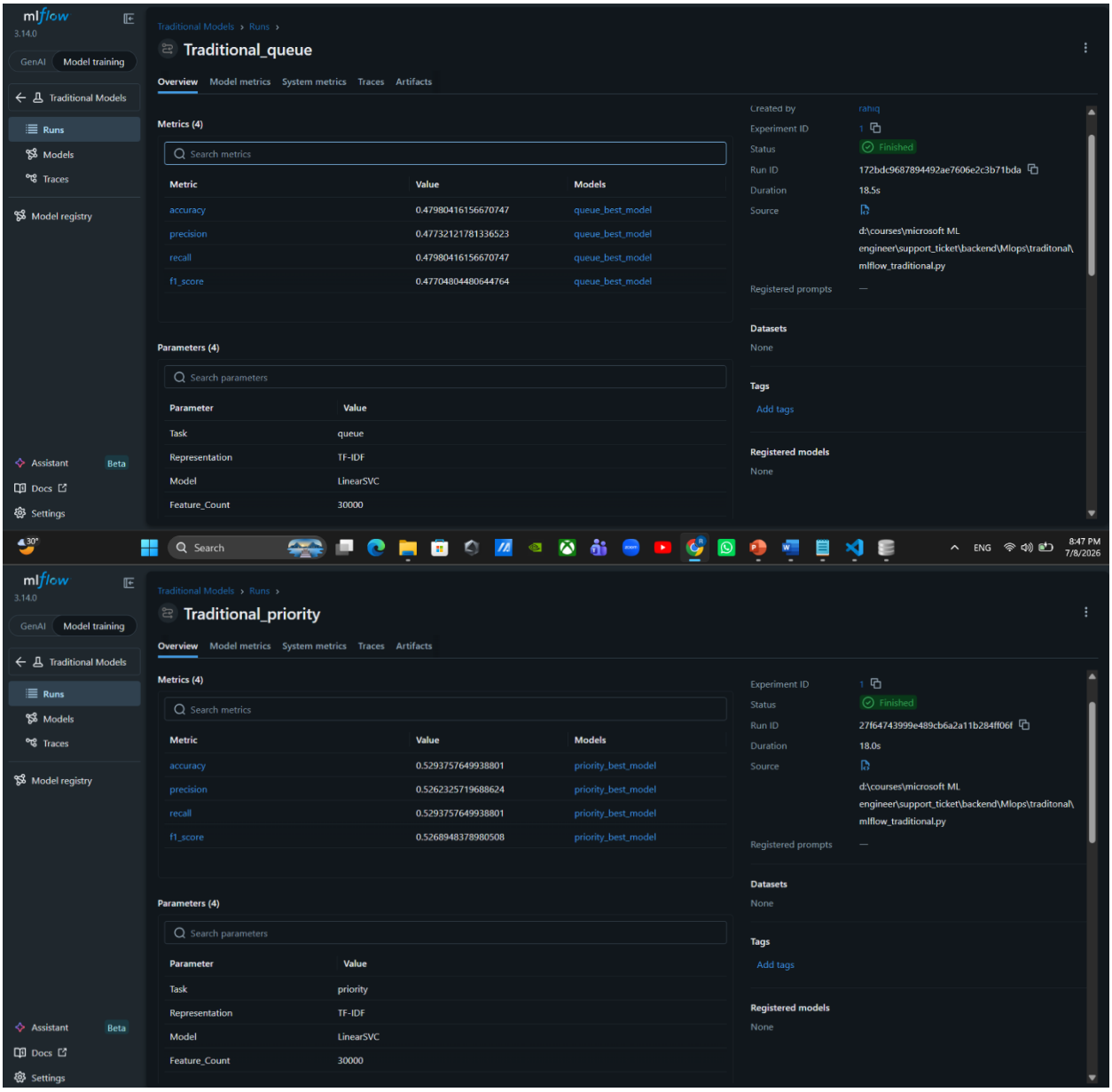
- Traditional Machine Learning
- DistilBERT
- Retrieval-Augmented Generation (RAG)

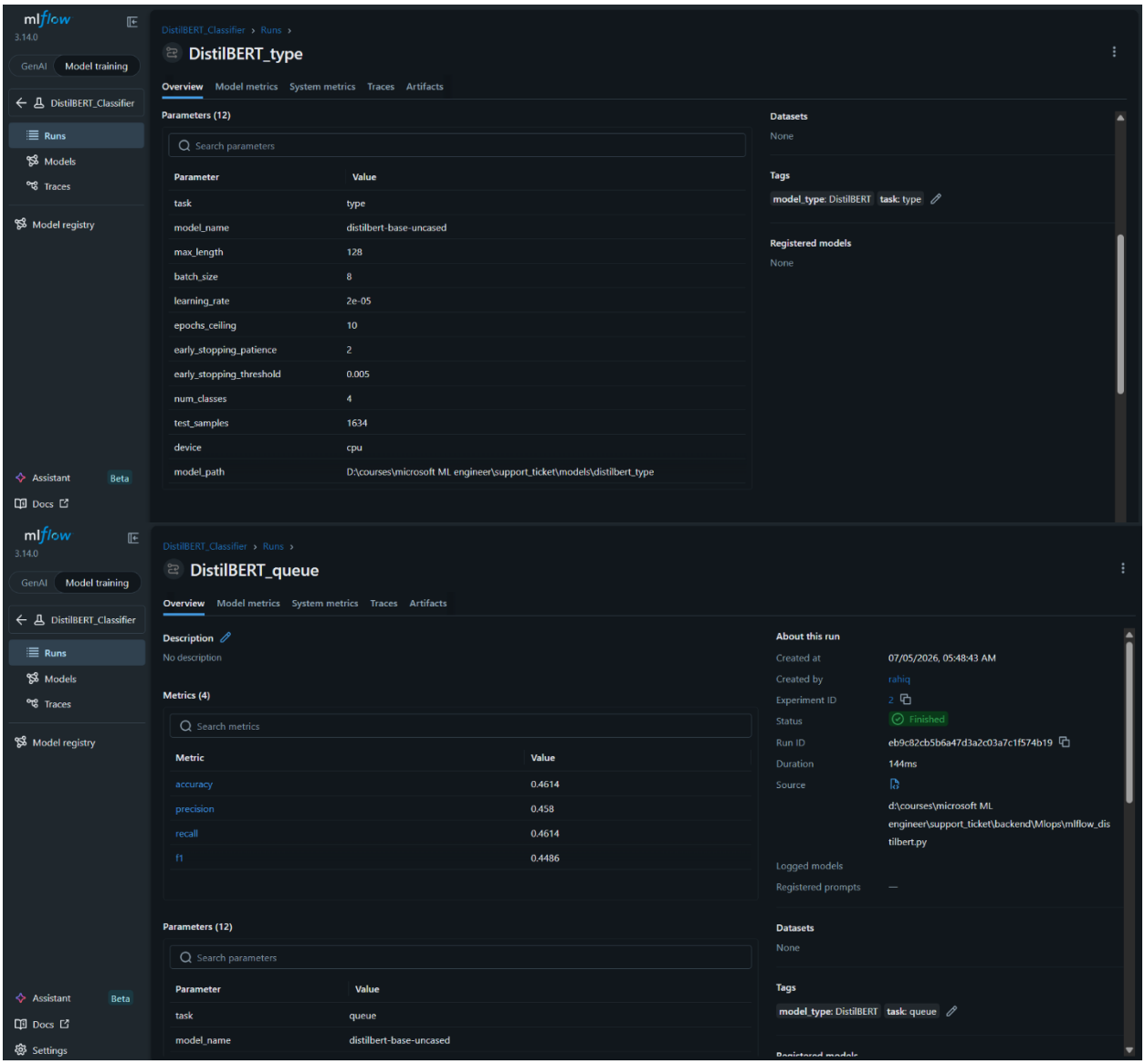
During every training run, MLflow records:

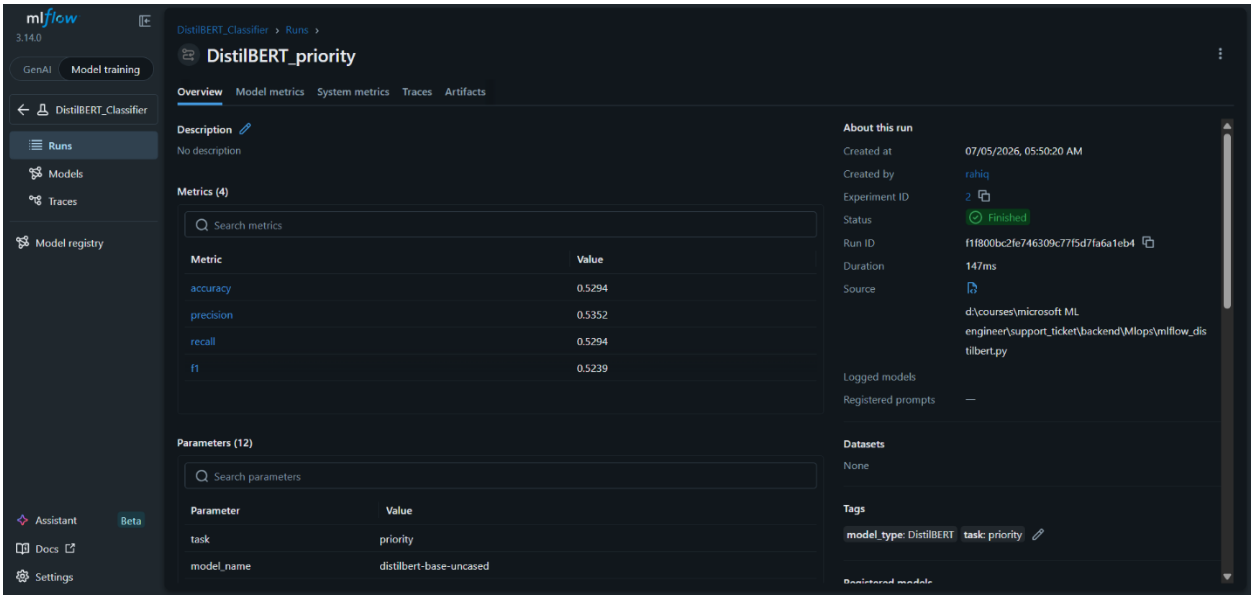
- Model parameters
- Training configuration
- Accuracy
- Precision
- Recall
- F1-score
- Classification reports
- Generated artifacts

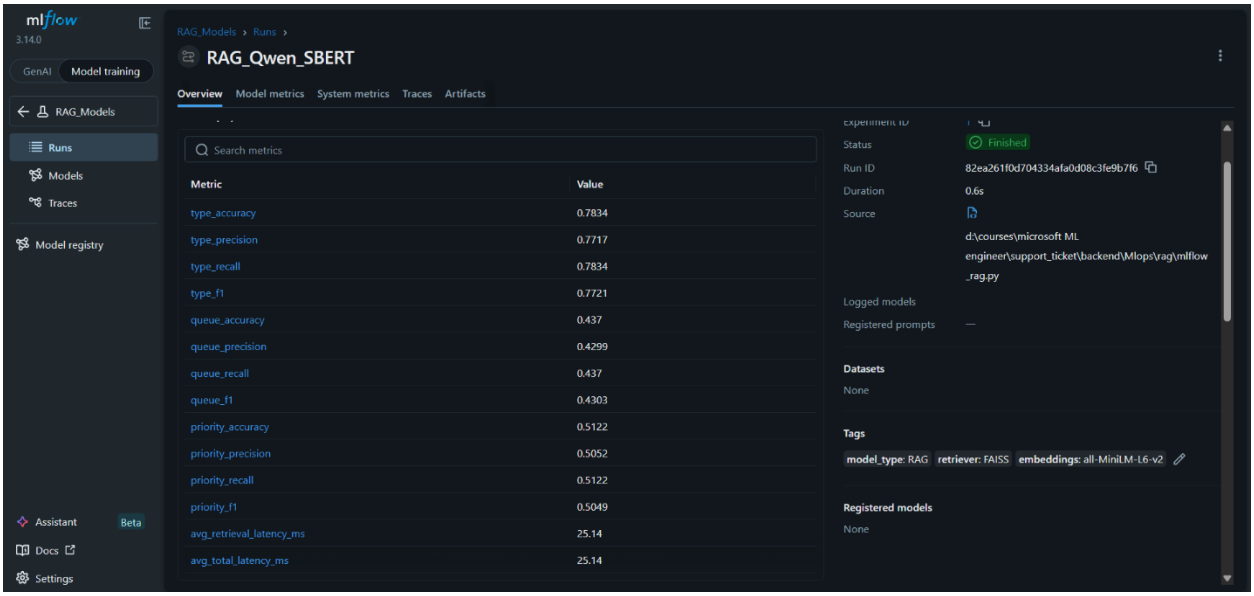
This allows experiments to be compared efficiently while preserving the complete history of model development.

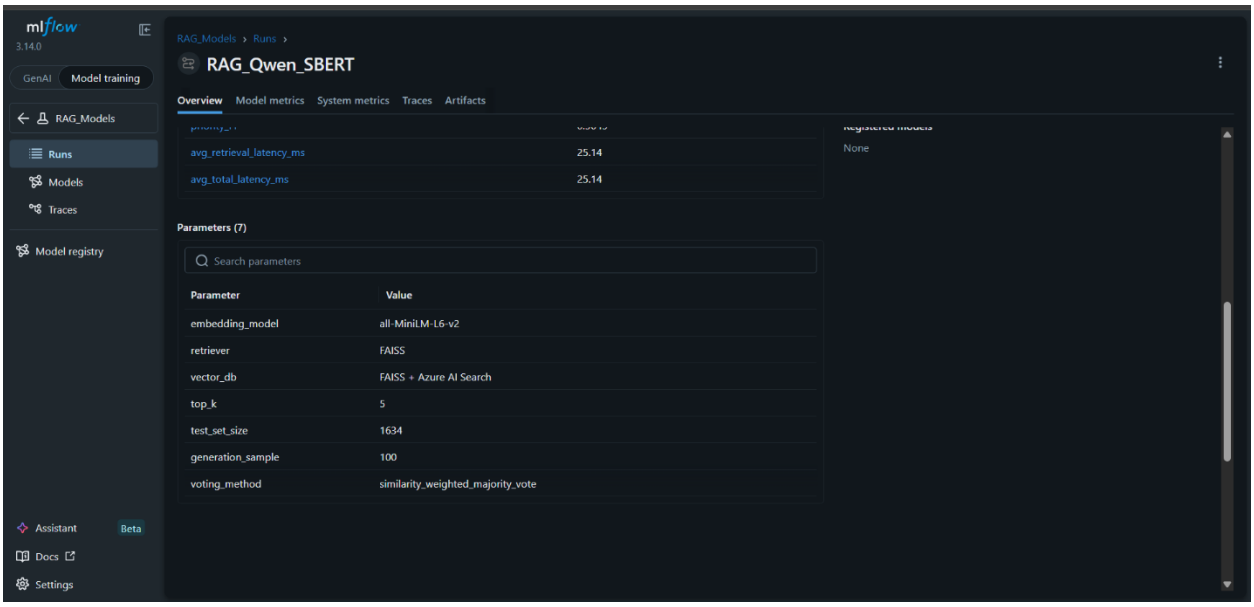



mlflow
3.14.0
GenAI
Model training
Traditional Models
Runs
Models
Traces
Model registry
Assistant
Beta
Docs
Settings
Traditional Models › Runs ›
Traditional_queue
Overview
Model metrics
System metrics
Traces
Artifacts
Metrics (4)
Search metrics
Metric
Value
Models
accuracy
0.47980416156670747
queue_best_model
precision
0.47732121781336523
queue_best_model
recall
0.47980416156670747
queue_best_model
f1_score
0.47704804480644764
queue_best_model
Parameters (4)
Search parameters
Parameter
Value
Task
queue
Representation
TF-IDF
Model
LinearSVC
Feature_Count
30000
Created by
rahiq
Experiment ID
1
Status
Finished
Run ID
172bdc9687894492ae7606e2c3b71bda
Duration
18.5s
Source
d:\courses\microsoft ML engineer\support_ticket\backend\Mlops\traditonal\mlflow_traditional.py
Registered prompts
—
Datasets
None
Tags
Add tags
Registered models
None
Search
ENG
8:47 PM
7/8/2026
mlflow
3.14.0
GenAI
Model training
Traditional Models
Runs
Models
Traces
Model registry
Assistant
Beta
Docs
Settings
Traditional Models › Runs ›
Traditional_priority
Overview
Model metrics
System metrics
Traces
Artifacts
Metrics (4)
Search metrics
Metric
Value
Models
accuracy
0.5293757649938801
priority_best_model
precision
0.5262325719688624
priority_best_model
recall
0.5293757649938801
priority_best_model
f1_score
0.5268948378980508
priority_best_model
Parameters (4)
Search parameters
Parameter
Value
Task
priority
Representation
TF-IDF
Model
LinearSVC
Feature_Count
30000
Experiment ID
1
Status
Finished
Run ID
27f64743999e489cb6a2a11b284ff06f
Duration
18.0s
Source
d:\courses\microsoft ML engineer\support_ticket\backend\Mlops\traditonal\mlflow_traditional.py
Registered prompts
—
Datasets
None
Tags
Add tags
Registered models
None


mlflow
3.14.0
GenAI
Model training
DistilBERT_Classifier
Runs
Models
Traces
Model registry
Assistant
Beta
Docs
DistilBERT_Classifier › Runs ›
DistilBERT_type
Overview
Model metrics
System metrics
Traces
Artifacts
Parameters (12)
Search parameters
Parameter
Value
task
type
model_name
distilbert-base-uncased
max_length
128
batch_size
8
learning_rate
2e-05
epochs_ceiling
10
early_stopping_patience
2
early_stopping_threshold
0.005
num_classes
4
test_samples
1634
device
cpu
model_path
D:\courses\microsoft ML engineer\support_ticket\models\distilbert_type
Datasets
None
Tags
model_type: DistilBERT
task: type
Registered models
None
mlflow
3.14.0
GenAI
Model training
DistilBERT_Classifier
Runs
Models
Traces
Model registry
Assistant
Beta
Docs
Settings
DistilBERT_Classifier › Runs ›
DistilBERT_queue
Overview
Model metrics
System metrics
Traces
Artifacts
Description
No description
Metrics (4)
Search metrics
Metric
Value
accuracy
0.4614
precision
0.458
recall
0.4614
f1
0.4486
Parameters (12)
Search parameters
Parameter
Value
task
queue
model_name
distilbert-base-uncased
About this run
Created at
07/05/2026, 05:48:43 AM
Created by
rahiq
Experiment ID
2
Status
Finished
Run ID
eb9c82cb5b6a47d3a2c03a7c1f574b19
Duration
144ms
Source
d:\courses\microsoft ML engineer\support_ticket\backend\Mlops\mlflow_distilbert.py
Logged models
Registered prompts
—
Datasets
None
Tags
model_type: DistilBERT
task: queue


mlflow
3.14.0
GenAI
Model training
DistilBERT_Classifier
Runs
Models
Traces
Model registry
Assistant
Beta
Docs
Settings
DistilBERT_Classifier › Runs ›
DistilBERT_priority
Overview
Model metrics
System metrics
Traces
Artifacts
Description
No description
Metrics (4)
Search metrics
Metric
Value
accuracy
0.5294
precision
0.5352
recall
0.5294
f1
0.5239
Parameters (12)
Search parameters
Parameter
Value
task
priority
model_name
distilbert-base-uncased
About this run
Created at
07/05/2026, 05:50:20 AM
Created by
rahiq
Experiment ID
2
Status
Finished
Run ID
f1f800bc2fe746309c77f5d7fa6a1eb4
Duration
147ms
Source
d:\courses\microsoft ML engineer\support_ticket\backend\Mlops\mlflow_distilbert.py
Logged models
Registered prompts
—
Datasets
None
Tags
model_type: DistilBERT
task: priority


mlflow
3.14.0
GenAI
Model training
RAG_Models
Runs
Models
Traces
Model registry
Assistant
Beta
Docs
Settings
RAG_Models › Runs ›
RAG_Qwen_SBERT
Overview
Model metrics
System metrics
Traces
Artifacts
Search metrics
Metric
Value
type_accuracy
0.7834
type_precision
0.7717
type_recall
0.7834
type_f1
0.7721
queue_accuracy
0.437
queue_precision
0.4299
queue_recall
0.437
queue_f1
0.4303
priority_accuracy
0.5122
priority_precision
0.5052
priority_recall
0.5122
priority_f1
0.5049
avg_retrieval_latency_ms
25.14
avg_total_latency_ms
25.14
Status
Finished
Run ID
82ea261f0d704334afa0d08c3fe9b7f6
Duration
0.6s
Source
d:\courses\microsoft ML engineer\support_ticket\backend\Mlops\rag\mlflow_rag.py
Logged models
Registered prompts
—
Datasets
None
Tags
model_type: RAG
retriever: FAISS
embeddings: all-MiniLM-L6-v2
Registered models
None




## 5.3 Monitoring Framework

Unlike offline evaluation, monitoring evaluates the performance of the deployed system using newly generated tickets stored in the database.

A monitoring module periodically evaluates several performance indicators and determines whether the deployed model continues to perform satisfactorily.

Rather than relying on a single metric, the monitoring framework combines multiple independent checks to provide a comprehensive assessment of model health.

## 5.4 Automated Retraining Checks

To determine when retraining is required, six independent monitoring conditions were implemented.

| Condition | Purpose |
|---|---|
| Average feedback rating | Detect declining customer satisfaction |
| Average AI confidence | Detect decreasing prediction confidence |
| Reopened ticket ratio | Detect tickets requiring human intervention |
| Failed ticket ratio | Detect AI failures |
| Ticket volume increase | Detect workload distribution changes |
| New ticket category detection | Detect emerging support requests |

## 5.5 Configurable Thresholds

All monitoring thresholds are defined in a dedicated configuration module, allowing system sensitivity to be adjusted without modifying the monitoring logic.

| Metric | Threshold |
|---|---|
| Average feedback rating | < 3.5 / 5 |
| Average confidence | < 0.65 |
| Reopened ticket ratio | > 15% |
| Failed ticket ratio | > 20% |
| Ticket volume increase | > 50% |

Using a centralized configuration improves maintainability and enables threshold tuning as system requirements evolve.

## 5.6 Retraining Alert Management

When one or more monitoring conditions are triggered, the system automatically creates a retraining alert.

Each alert stores:

- Triggered condition
- Severity level
- Measured value
- Threshold value
- Creation time
- Resolution status

To prevent duplicate notifications, the system maintains only one active alert for each condition. Once the monitored metric returns to an acceptable range, the corresponding alert is automatically resolved.

This mechanism ensures that administrators receive meaningful notifications without unnecessary repetition.

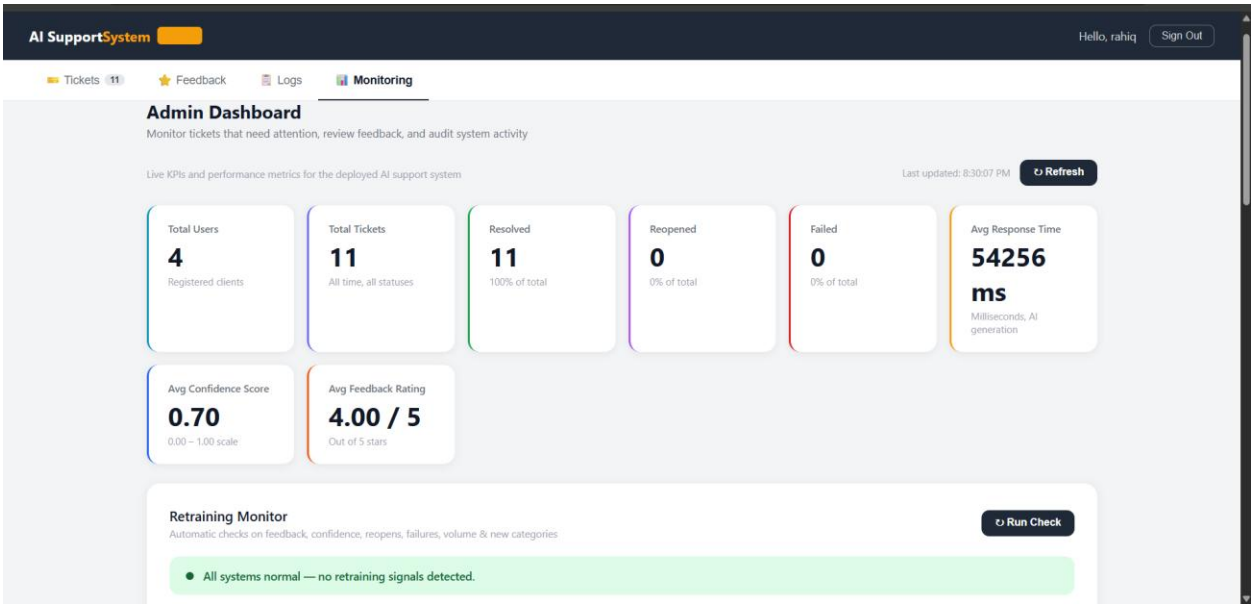
## 5.7 Overall System Status

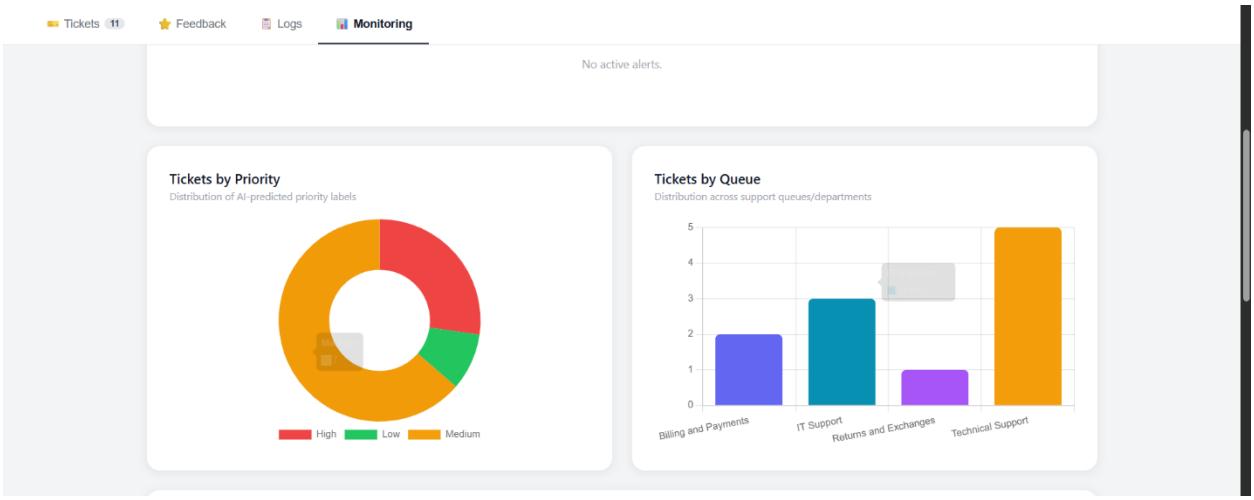
After evaluating all monitoring conditions, the system determines an overall operational status.

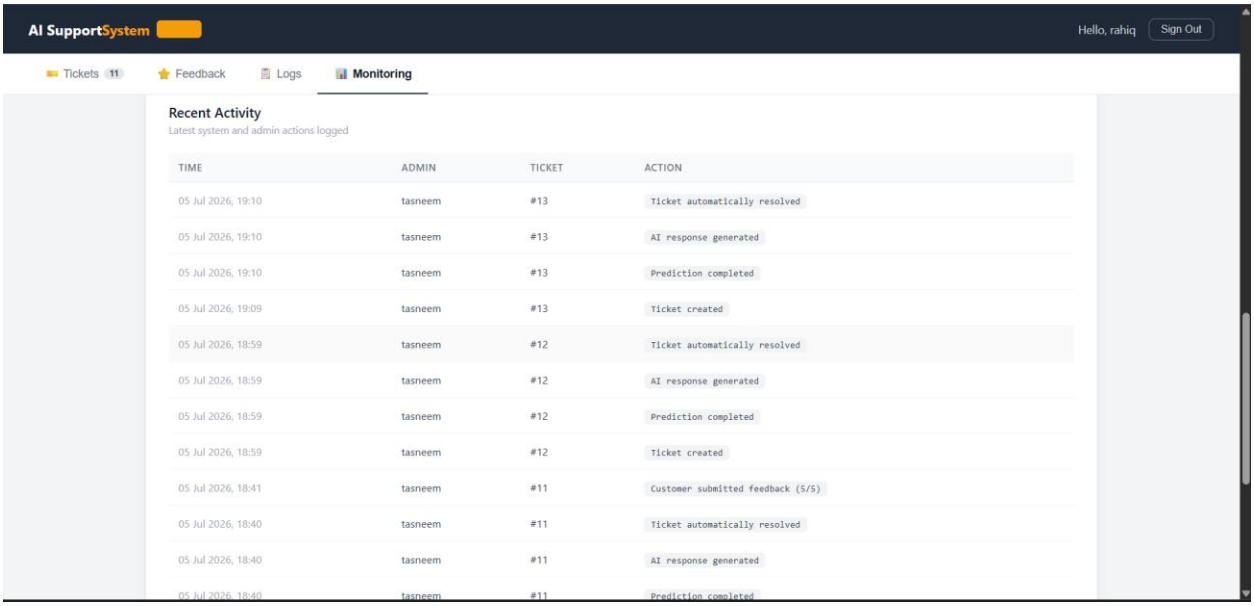
Possible states include:

| Status | Description |
|---|---|
| OK | No monitoring condition is triggered |
| Warning | One or more non-critical conditions are triggered |
| Retraining Recommended | Critical conditions indicate that model retraining should be performed |

This provides administrators with a concise summary of the deployed model's health.






## 6 – Future Work

Although the proposed system successfully automates support ticket classification and response generation, several improvements can further enhance its capabilities and scalability.

Future work may include:

- **Multilingual Support:** Extend the system to support additional languages, allowing organizations to serve customers from different regions using the same platform.
- **Expanded Ticket Categories:** Train the classification models on larger and more diverse datasets to support additional ticket types and specialized support domains.
- **Cloud Database Integration:** Replace the SQLite database with cloud-based database services such as Azure SQL Database or PostgreSQL to improve scalability, reliability, and concurrent access.
- **Continuous Learning:** Implement automated retraining pipelines that periodically update the deployed models using newly collected tickets and customer feedback.

These enhancements would further improve the scalability, adaptability, and practical deployment of the intelligent support ticket system in real-world customer support environments.

# 7 – Conclusion

This project presented the design and implementation of an **Intelligent Support Ticket Classification System with Retrieval-Augmented Generation (RAG)** that automates multiple stages of the customer support workflow.

The project began with extensive data preprocessing, including exploratory data analysis, text cleaning, feature engineering, semantic duplicate removal, and leakage prevention to ensure reliable model evaluation. Multiple machine learning approaches were then developed and compared, including traditional TF-IDF classifiers, a fine-tuned DistilBERT model, and a Retrieval-Augmented Generation pipeline.

Experimental results demonstrated that the **TF-IDF + LinearSVC** approach achieved the highest performance for ticket routing tasks, particularly for queue and priority prediction, while **DistilBERT** provided the strongest performance for ticket type classification. Although the retrieval-based approach produced lower classification accuracy, it significantly improved response generation by incorporating relevant historical support cases through semantic retrieval.

The trained models were successfully deployed using **Azure Machine Learning**, while **Azure AI Search** enabled efficient retrieval of semantically similar tickets. These components were integrated into a **FastAPI** backend with secure authentication, a relational database, and dedicated client and administrator dashboards, resulting in a complete end-to-end intelligent customer support platform.

Finally, MLOps practices were incorporated through **MLflow experiment tracking**, continuous monitoring, and automated retraining recommendations, improving the maintainability and long-term reliability of the deployed system.

Overall, the project demonstrates how machine learning, semantic retrieval, large language models, cloud deployment, and MLOps can be combined to build an effective AI-powered customer support solution capable of improving response quality, reducing manual effort, and supporting scalable customer service operations.